

Admin & Organizer Interface Audit

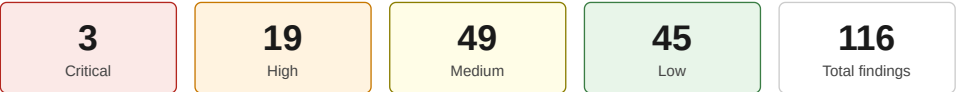
Application: Playora (rush-x.xyz) · Artifact reviewed: `index.html` (compiled Expo web bundle, commit 01d228f) · Date: 13 September 2026

Purpose. Full audit of the administrator and organizer interfaces for bugs, dead ends, ambiguities and non-compliance with standard internal controls, plus a senior-architect review of security, architecture, code quality and deployment readiness.

Method. The shipped production bundle was split into its 2,514 Metro modules, the route table was recovered, and every admin and organizer screen was read alongside the backend functions it calls. Findings cite `<module>.js: <line>` in the prettified module dump so each item can be located in the bundle (a module-to-route map is in the appendix).

How to read. Section 3 is the findings register in the requested format (Risk Area, Identified Gap, Potential Impact, Recommended Remediation). Sections 4-6 give the interface-by-interface status, the architecture review, and a prioritised remediation plan.

1. Executive summary



Bottom line. The deployed file is a working front-end prototype whose "backend" is an in-browser mock store persisted to `localStorage`. The admin and organizer interfaces are rich and mostly functional as UI, but none of their controls (role checks, approvals, sanctions, refunds, commission settings, audit logging) are enforceable, because they execute in the user's own browser. In the shipped build there is also no legitimate way to obtain an admin account, demo mode is permanently active, and the Railway deployment configuration cannot build. The report therefore separates *fix-in-place* defects (screen bugs and dead ends, section 3) from *architectural* gaps that require a real server (section 5).

Scope note. The repository contains only the compiled bundle. Every fix below has to be applied in the original Expo project and re-exported; line references point into the prettified module dump described in Appendix A.

Key themes across the admin and organizer interfaces

- Controls exist in the UI but not in a place where they can be enforced.** Role checks (`sn`, `Ms`, `on`, `organizer-of-game`) are applied consistently in the mock backend, which is good design, but they run in the visitor's browser (F-XC-1, F-ADM1-33).
- Segregation of duties is missing.** An admin can submit and approve their own organizer application (F-ADM1-1) and register, approve and set the commission of their own venue (F-ADM1-19). An organizer can sanction any user in the platform, not only participants of their own matches (F-ORG2-8).
- Decisions do not require or preserve a rationale.** Reject/suspend back-fill a canned reason (F-ADM1-2), weight and rule changes are logged without before/after values or a reason (F-ADM2-1), and cancel/kick reasons are optional (F-ORG2-23).
- The audit trail cannot be relied on.** It is a 500-row ring buffer in `localStorage`, and read-only admin screens append to it on every visit, evicting real evidence and silently truncating the statistics computed from it (F-XC-7, F-ADM2-2).
- Dead ends in the organizer onboarding loop.** "Request info" leaves an application in permanent limbo because the applicant cannot resubmit (F-ADM1-4, F-ORG1-11); suspended organizers never see the reason (F-ORG1-12); identity documents are a self-attested toggle (F-ORG1-14); `/organizer/new` is an orphaned third creation wizard (F-ORG1-1).
- Financial controls have gaps.** Cancelling a series does not refund paid seats (F-ORG2-1); an in-progress match can be cancelled with forced full refunds (F-ORG2-11); series revenue is estimated from list price rather than payments (F-ORG2-7); exported CSVs have no formula-injection guard (F-ADM2-15).
- Errors are swallowed into success-looking states.** Across both areas, `catch {}` maps failures to empty lists, so "No fraud signals", "No funnel events" or "No recommendations" can mean "the request failed" (F-XC-14, F-ADM2-6, F-ORG1-20, F-ORG2-17).
- Audience "world" partitioning is invisible to admins.** Applications, players and incidents from the other audience are silently hidden, and stats do not reconcile with the visible list (F-ADM1-9).

Highest-priority items

- F-XC-1** (Critical, Platform (all interfaces)): The entire "backend" runs inside the browser; every admin and organizer control is client-side
- F-XC-5** (Critical, Sign-in, organizer/apply, admin/organizers): Demo mode is permanently on in this build (static OTP, seeded admins, auto-approved organizer)
- F-XC-7** (Critical, Platform (admin actions, sanctions, approvals)): Audit trail is stored in the user's browser, capped at 500 rows, and contains placeholder values
- F-ADM1-1** (High, admin/application/[id]): Admin can approve their own organizer application (no self-review guard)
- F-ADM1-19** (High, admin/venues (and venue/apply)): Admin can register a venue, approve it and set its own commission (no owner ≠ reviewer check, no role gate on apply)
- F-ADM1-32** (High, all five screens (cross-cutting)): Admin audit trail is a client-side, 500-entry `localStorage` ring buffer with a hard-coded "ip" field
- F-ADM1-33** (High, all five screens (cross-cutting)): The admin check (`sn`) trusts a client-held profile role
- F-ADM1-4** (High, admin/application/[id] (and organizer/apply)): "Request info" is a dead end — applicant cannot resubmit and the queue shows no change
- F-ADM1-9** (High, admin/organizers (also admin/players, admin/conduct)): Applications (and players/incidents) from the other audience "world" are silently hidden from the admin
- F-ADM2-1** (High, admin/concierge, admin/demand, admin/optimizer): Weight/rule changes are audited without reason, before/after values or a target
- F-ADM2-2** (High, admin/awards, admin/insights, admin/funnel, admin/concierge): Read-only screens write to a 500-entry FIFO audit log, evicting admin evidence and truncating stats
- F-ORG1-11** (High, organizer/apply): "Request info" from the reviewer is a dead end for the applicant
- F-ORG1-14** (High, organizer/apply): Identity document "upload" is a self-attested toggle; document is never collected
- F-ORG2-1** (High, organizer/series/[id]): Cancelling a series cancels all upcoming occurrences without refunding paid seats

Findings by risk area

Risk area	Count
Ambiguity	21
Data integrity	15
UX dead end	14

Findings by primary screen

Interface	Count
organizer/match	15
Platform-wide	12
admin/application	8

Audit trail	13
Authorization	11
Validation	9
Financial control	8
Availability	8
Bug/crash	8
Privacy	6
Segregation of duties	3

organizer/series	8
organizer/create	8
organizer/apply	7
admin/players	7
admin/venues	6
admin/conduct	6
admin/insights	6
admin/organizers	5
admin/concierge	5
admin/awards	4
organizer/index	4
admin/demand	3
organizer/new	3
Sign-in / auth	2
admin/feed	2
(tabs)/profile	2
admin/optimizer	1
admin/funnel	1
organizer/quick	1

2. How the application actually works (architecture snapshot)

Runtime

- **Stack.** Expo Router / React Native Web, compiled by Metro into one script and embedded in a single HTML file with all fonts and images as `data:` URIs. Routes live in the Expo Router context module (id 18); 87 route files, 12 admin and 9 organizer screens.
- **Data layer.** Screens call a facade (module 671) which forwards to `store.mockX`. The store (672) is a Proxy that first tries `POST {backend}/rpc {fn, args}` and falls back to the local mock (631) on any error. The backend base URL defaults to `http://localhost:8090` and is overridable from `localStorage`.
- **Persistence.** ~60 collections (users, profiles, games, payments, sanctions, applications, audit log, ...) are JSON-serialised into `localStorage` keys named `playora.mock.*.v1`. Every mutation rewrites the whole collection; the whole database is hydrated at start-up.
- **Auth.** `AuthProvider` (630) loads a session from `localStorage["secure.playora_session"]`, otherwise a guest session. Email/password and phone-OTP flows exist both locally (631) and remotely (913). Roles are a `role` field on the profile row: `user`, `organizer`, `analyst`, `admin`.
- **Authorization.** Backend-style checks exist and are consistently applied in the mock: `sn(userId)` asserts admin, `Ms(userId)` asserts admin-or-analyst, `on(userId)` asserts approved organizer, and per-game checks compare `organizer_id`. These are good foundations, but they run client-side.

Consequences for this audit

- The CSP (`connect-src 'none'`) guarantees the remote path is never taken from the deployed page, so **everything the admin or organizer does stays on their own device**.
- "Demo world" (`demoWorldEnabled()`) is derived from whether `script` is available, which it never is in a browser, so seeded admins, the static OTP code and the auto-approved organizer are always present.
- The audit log, lockout counters, OTP throttles and role table are all user-editable.
- The screens themselves are well-structured (loading/empty/error states, i18n in en/ar, accessibility labels), so most UI-level findings are targeted fixes rather than rewrites.

Call path of every screen action

1. Screen calls `fetchX()` on the facade (671).
2. The store proxy (672) intercepts `mockX` and calls `rpc()` (673).
3. `ensureBackend()` reads `localStorage.playora.backend[.url]` and probes `{base}/health`.
4. Remote and healthy: `POST {base}/rpc {fn, args}` with a Bearer token. Otherwise, or on any network error (always the case under the shipped CSP): run `mockX` locally in 631.
5. The mock reads and writes JSON arrays in `localStorage.playora.mock.*.v1`; write failures are logged to the console only.

3. Findings register

Sorted by severity. IDs: XC = cross-cutting/platform, ADM1 = admin governance screens, ADM2 = admin analytics/config screens, ORG1 = organizer onboarding/creation, ORG2 = organizer match/series management.

ID / Severity	Interface	Risk Area	Identified Gap (with evidence)	Potential Impact	Recommended Remediation
F-XC-1 Critical	Platform (all interfaces)	Authorization	The entire "backend" runs inside the browser; every admin and organizer control is client-side Every <code>fetchX/mutateX</code> call resolves to an in-memory mock store persisted to <code>localStorage</code> . Role checks such as <code>sn(e)</code> (admin) and <code>on(e)</code> (approved organizer) run in the same browser tab as the user. A user can open DevTools, edit <code>playora.mock.profiles.v1</code> to set <code>role: "admin"</code> , and unlock every admin screen; they can equally edit games, payments, sanctions and the audit log. Evidence: 672.js:37-48 — <code>store = new Proxy(e, { get(...) { ... rpc(String(n), e, () => c(...e)) } });</code> ; 673.js:71-88 — <code>if (await k(), "remote" !== o) return l(); ... catch { return ({o = "mock"}, l()); }</code> ; 631.js:467-538 — <code>q = "playora.mock.users.v1", Y = "playora.mock.profiles.v1", ...</code> ; 618.js — <code>storage</code> is <code>AsyncStorage</code> (<code>localStorage</code> on web)	No control in the admin or organizer interfaces is enforceable. Approvals, sanctions, commissions, refunds and audit records can be forged by any visitor. Nothing is shared between devices, so an admin decision on one laptop is invisible on another.	Treat the current build as a prototype. Stand up a real API (the <code>rpc</code> transport in 673.js already expects <code>POST /rpc {fn, args}</code> with a Bearer token) and move the mock store's rules server-side with database-backed persistence. Until then label the deployment as a demo and do not onboard real organizers or admins.
F-XC-5 Critical	Sign-in, organizer/apply, admin/organizers	Authorization	Demo mode is permanently on in this build (static OTP, seeded admins, auto-approved organizer) Because <code>script</code> is unavailable in the browser, <code>demoWorldEnabled()</code> is always true. Phone sign-in accepts the fixed code 123456 and even prints it on screen. The first user to sign in is silently given an approved organizer application, attributed to "Platform Admin" as reviewer, and promoted to <code>role: "organizer"</code> . Seed profiles "Platform Admin", "Platform Admin (سيّدات)" and "Platform Analyst" are inserted into the profile table. Evidence: 632.js:83 — <code>serverGradeHashing = !o</code> where <code>o = t.scrptHex</code> (undefined in the browser); 631.js:1682 — <code>Fa = () => !c.serverGradeHashing ...</code> ; 631.js:753-758 — <code>sa = "123456" ... return !c.serverGradeHashing e ? sa : randomSixDigit();</code> ; 631.js:800 — <code>return { sent: !0, demo_code: a }</code> ; 613.js:137-141 — renders <code>E("otpDemoHint", { code: U })</code> ; 631.js:4655-4686 — <code>fn</code> inserts an application with <code>status: "approved"</code> , <code>reviewed_by: oe</code> and calls <code>mn(e, "organizer")</code> ; 631.js:4103-4110 — <code>an()</code> calls <code>fn(t)</code> for the signed-in user	The organizer vetting control (ID document, phone hash, admin review) is bypassed for the very user most likely to test it; a fabricated approval record with a fake reviewer pollutes the audit trail; OTP login provides no authentication.	Gate demo behaviour on an explicit build flag (<code>EXPO_PUBLIC_DEMO=1</code>) rather than on crypto capability, default it to off, and never ship seeded privileged profiles or static OTP codes in a public build.
F-XC-7 Critical	Platform (admin actions, sanctions, approvals)	Audit trail	Audit trail is stored in the user's browser, capped at 500 rows, and contains placeholder values Every privileged action is "audited" into <code>localStorage</code> of the actor's own browser, truncated to the 500 most recent rows, with a literal string "server-captured" instead of an IP address. The actor can edit or delete the log. No screen reads it for compliance purposes (<code>readAudit</code> is used only to compute funnel/concierge stats at 631.js:10944, 11715, 15169). Evidence: 643.js:9-15 — <code>logAdminAudit</code> writes <code>{ applicant, device: c(), ip: "server-captured" }</code> where <code>c()</code> returns "web"; 643.js:25-26 — <code>n = "playora.audit.v1", o = 500</code> ; 643.js:33-44 — <code>u.unshift(...); setItem(n, JSON.stringify(u.slice(0, o)))</code> ; 643.js:47-48 — <code>appendAudit</code> hook is optional and absent on web	No non-repudiation for approvals, sanctions, refunds, commission changes or role changes. Fails basic internal-control expectations for an admin console.	Emit audit events server-side (append-only table with actor, target, before/after, IP, user agent, timestamp) and add an admin "Audit log" screen with filtering and export.
F-ADM1-1 High	admin/application/[id]	Segregation of duties	Admin can approve their own organizer application (no self-review guard) An admin can submit an organizer application from the normal apply flow and then approve it (or approve a colleague's) from this screen. <code>mockGrantCrossPartition</code> (631.js:4348) does have a <code>e === t</code> self-check, so the pattern exists but was not applied here. Evidence: 631.js:4572-4576 — <code>r.mockReviewApplication = async (e, t, a, i) => { (await ei(), await sn(e)); const n = Qt.applications.find((e) => e.id === t); ... await _n(e, un(n.user_id), ...) (no n.user_id !== e check);</code> 631.js:4456-4475 <code>mockSubmitOrganizerApplication</code> has no role restriction; 2459.js contains no admin gate (grep for "admin"/"role" returns nothing); 631.js:4437-4442 <code>mn</code> only skips the role write for admins, but 631.js:4580 still sets <code>n.status = "approved"</code> and 631.js:4290 <code>rn = (e) => "approved" === nn(e)?.status</code> then grants organizer powers.	Admin self-elevation to organizer bypasses identity/risk review; risk flags (duplicate document, velocity) are irrelevant when reviewer = applicant.	In <code>mockReviewApplication</code> , throw (e.g. <code>E_YOU_CANNOT_REVIEW_YOUR_OWN_APPLICATION</code> when <code>n.user_id === e</code> , and log the attempt; optionally block the apply flow for <code>role === "admin"</code> .
F-ADM1-19 High	admin/venues (and venue/apply)	Financial control	Admin can register a venue, approve it and set its own commission (no owner ≠ reviewer check, no role gate on apply) A single admin account can create a venue it owns, approve it, set commission to 0 %, and collect 100 % of gross via <code>net_to_venue_kwd</code> . No four-eyes control and no self-review guard. Evidence: 631.js:6578-6626 — <code>mockApplyVenue</code> has no role check (<code>owner_id: e, status: "pending", commission_value: 10</code>); 2503.js has no admin gate; 631.js:6704-6710 — <code>mockAdminReviewVenue: (await ei()), await sn(e); const n = hr(t); ... n.status = "approve" === a ? "approved" : ... (no n.owner_id !== e check);</code> 631.js:6737-6742 — <code>mockAdminSetCommission</code> sets <code>commission_value = Math.max(0, Math.min(100, i))</code> with no owner check; 631.js:6854 — settlements use the venue's <code>commission_type/value</code> snapshotted on the booking.	Direct revenue leakage; commission fraud with a clean-looking audit trail (the log records the change but not that reviewer = owner).	Throw when <code>n.owner_id === e</code> in review/commission mocks, require a second admin (or a reason + threshold) for commission below a floor, and log previous values.

ID / Severity	Interface	Risk Area	Identified Gap (with evidence)	Potential Impact	Recommended Remediation
F-ADM1-32 High	all five screens (cross-cutting)	Audit trail	Admin audit trail is a client-side, 500-entry localStorage ring buffer with a hard-coded "ip" field Every privileged decision in scope (approve/reject/suspend organizer, venue decisions, commission changes, sanction reviews, rating adjustments) is recorded only in the actor's own browser storage, truncated to 500 rows, with a literal placeholder for IP and a non-unique 6-char actor reference. The record can be cleared by the same user it is meant to hold accountable. This is inherent to the mock architecture; it needs confirmation whether a server store replaces it before launch. Evidence: 643.js:9-14 — <code>logAdminAudit = async (e, t, n, o) => await l(e, s(t), Object.assign({ applicant: s(n) ?? "unknown", device: c(), ip: "server-captured" }, o ?? {}));</code> 643.js:25, 34-43 — <code>key "playora.audit.v1", u.slice(0, o)</code> with <code>o = 500</code> ; 643.js:44-46 — write failures only <code>console.error</code> ; 643.js:29-31 — <code>actorRef</code> keeps only the last 6 chars of the admin id; 643.js:48-49 — optional <code>appendAudit</code> hook.	No reliable, tamper-evident audit trail for any admin action; investigations cannot attribute actions.	Persist admin audit events server-side via <code>appendAudit</code> (make it mandatory for <code>logAdminAudit</code>), store full actor id, capture IP/U/ server-side, and remove the cap for admin events
F-ADM1-33 High	all five screens (cross-cutting)	Authorization	The admin check (sn) trusts a client-held profile role Authorization is evaluated against data the client controls and the "caller id" is whatever the screen passes; nothing binds it to the authenticated session. Adequate for a demo store, not for production — needs confirmation of the intended backend. Evidence: 631.js:4300-4306 — <code>async function sn(e) { if ((await ei()), "admin" !== Qt.profiles.find((t) => t.id === e)?.role)) throw ... }</code> ; <code>Qt.profiles</code> is hydrated from local storage (631.js:1737-1760 <code>ei/ti</code>) and every mock takes the caller id as a plain argument (e.g. 671.js:456 <code>{o, c, s, n} => t.store.mockReviewApplication(o, c, s, n)</code>).	Any user can obtain admin behaviour by editing stored profile role or calling the facade with an admin id.	Enforce role checks server-side from the session principal; never accept <code>adminId</code> from the request body. ---
F-ADM1-4 High	admin/application/[id] (and organizer/apply)	UX dead end	"Request info" is a dead end — applicant cannot resubmit and the queue shows no change After the admin requests more information, the applicant receives a notification (<code>wn(..., "info_requested", t)</code> , 631.js:4607) but any attempt to resubmit is rejected because the status is still <code>under_review</code> . On the admin side the card keeps the same "Under review" badge (1832.js:388-391, 1823.js:76) so the reviewer cannot tell that info was requested or whether it arrived. Evidence: 631.js:4600-4607 — <code>"request_info" === a</code> sets <code>n.status = "under_review"</code> (unchanged) and only writes <code>admin_notes</code> ; 631.js:4473 — <code>if ("under_review" === d?.status) throw new Error("E_YOUR_APPLICATION_IS_ALREADY_UNDER_REVIEW")</code> ; the only application-writing mocks are <code>mockSubmitOrganizerApplication</code> (4456) and <code>mockReviewApplication</code> (4572) — there is no "update/supplement application" mock.	Applications enter a permanent limbo; the admin's only exits are approve or reject without the requested information.	Introduce a distinct status (e.g. <code>info_requested</code>), that the submit mock accepts for resubmission (<code>d.status === "info_requested" → allowed, keep id</code>), add it to the filter chips (1832.js:478-484) and status-tone maps, and clear <code>admin_notes</code> or mark it answered on resubmission.
F-ADM1-9 High	admin/organizers (also admin/players, admin/conduct)	Availability	Applications (and players/incidents) from the other audience "world" are silently hidden from the admin The approval queue, player intel and incident queue are partitioned by the reviewing admin's own audience. A male-world admin never sees female applicants/players/sanctions (and vice-versa) unless a time-limited cross-partition grant exists (631.js:4307-4315), and the screens give no indication that records were withheld. Whether this is intended policy needs confirmation, but the silent behaviour is not. Evidence: 631.js:4562 — <code>cn(e, Qt.applications, (e) => un(e.user_id), "admin.applications")</code> ; 631.js:4316-4330 — <code>cn</code> returns <code>r</code> (the filtered subset) with no marker when the admin has no grant; 631.js:735-742 — <code>aa</code> defaults a profile with no <code>audience</code> to <code>"male"</code> ; same pattern at 631.js:5411 (player intel), 4401 (find players), 12234 (incident queue). 1832.js:337-342 shows "Queue is clear" when the filtered list is empty.	Applications and pending bans in the other world may go unreviewed indefinitely; the "Queue is clear" empty state is misleading; stats cards (see F-ADM1-10) will not reconcile with the list.	Either ensure at least one admin per world (and show which world the queue is for), or show a banner "N records hidden — other world" using <code>t.length - r.length</code> from <code>cn</code> , and surface the grant state in the header.
F-ADM2-1 High	admin/concierge, admin/demand, admin/optimizer	Audit trail	Weight/rule changes are audited without reason, before/after values or a target Platform-wide ranking/automation parameters are changed with an audit record that contains no reason, no changed key, no old/new value and <code>applicant: "unknown"</code> . Every stepper tap on the three screens produces an identical, uninformative entry. Evidence: 631.js:11709 — <code>await (0, w.logAdminAudit)("concierge.rules_changed", e, null, {})</code> ; 631.js:12362 — <code>logAdminAudit("demand.weights_changed", e, null, {})</code> ; 631.js:12562 — <code>logAdminAudit("optimizer.weights_changed", e, null, {})</code> ; 643.js:9-14 — <code>logAdminAudit</code> only adds <code>applicant: s(n) ?? "unknown", device, ip</code> to <code>o ?? {}</code> .	An investigator cannot reconstruct what an admin changed or why; a malicious or mistaken change to demand/optimizer/concierge behaviour is unrecoverable from the log.	Have <code>mockSet*Weights/mockSetConciergeRules</code> compute a diff (<code>{key, from, to}</code>) and pass it in the meta; require a <code>reason</code> string from the screer (prompt once per session or on save) and reject empty reasons.
F-ADM2-2 High	admin/awards, admin/insights, admin/funnel, admin/concierge	Audit trail	Read-only screens write to a 500-entry FIFO audit log, evicting admin evidence and truncating stats The audit store keeps only the newest 500 records, and purely read-only admin views (fraud list, BI dashboard, every filter change) append records each time. Meanwhile funnel "Total" and concierge learning stats are derived from the same rolling window, so they silently stop being totals once the log wraps. Evidence: 643.js:25 — <code>o = 500 ...</code> 643.js:43 <code>u.slice(0, o)</code> ; 631.js:14938-14941 — <code>mockGetAwardFraudSignals</code> logs <code>award.fraud_flagged</code> for every flagged match on every visit; 631.js:10666-10669 and 10981 — <code>bi.accessed</code> logged on every dashboard fetch (each filter tap in 1830.js:107/123/127 triggers <code>ee</code>); 631.js:15169 — funnel totals are computed from <code>readAudit()</code> ; 631.js:11716 — concierge <code>plansGenerated/autoInvites</code> also from <code>readAudit()</code> .	A few dozen dashboard interactions push <code>concierge.rules_changed</code> , <code>award.revoked</code> etc. out of the log; funnel/concierge figures under-report without any indicator.	Do not log read access into the same bounded store as privileged actions (or raise the cap and partition by type); dedupe <code>award.fraud_flagged</code> per match/day; derive funnel and concierge counters from a dedicated counter table rather than the audit log.

ID / Severity	Interface	Risk Area	Identified Gap (with evidence)	Potential Impact	Recommended Remediation
F-ORG1-11 High	organizer/apply	UX dead end	"Request info" from the reviewer is a dead end for the applicant The reviewer can ask for more information but the applicant has no way to provide it (no resubmission, no message channel), and the backend explicitly blocks resubmission while under review. The status card also labels the admin's free-text <code>admin_notes</code> as an information request, although the same field is written on <code>approve/reject/suspend</code> as a generic "Reviewer note" (631.js:4578-4583, 909.js:1157 <code>adminNotesLabel</code>: "Reviewer note"). Evidence: 631.js:4600-4607 — <code>request_info</code> keeps <code>status = "under_review"</code> and stores the note in <code>admin_notes</code>; 2459.js:476-491 — for <code>under_review/submitted</code> the screen renders "additionalInfoRequested" + <code>admin_notes</code> but the form is hidden (<code>ze = !F Re</code>, 2459.js:44; form only when <code>ze</code>, 2459.js:69-77) and no edit/resubmit control is rendered for that status (2459.js:493-501 only handles approved/rejected); 631.js:4472 — <code>if ("under_review" === d?.status) throw new Error("E_YOUR_APPLICATION_IS_ALREADY_UNDER_REVIEW")</code>.	Onboarding stalls indefinitely; admins wait for information that cannot be sent; internal reviewer notes may be shown to the applicant as if they were a request.	Introduce an explicit <code>info_requested</code> status that re-opens the form (pre-filled) and allows one resubmission that resets it to <code>under_review</code> ; keep a separate applicant-facing message field from internal <code>admin_notes</code> .
F-ORG1-14 High	organizer/apply	Validation	Identity document "upload" is a self-attested toggle; document is never collected No file picker, no upload, no requirement that the flag be true. The "secure storage" note (2459.js:263-275) implies a document is stored when nothing is. Evidence: 2459.js:241-262 — <code>onPress: () => he((e) => !e)</code> toggles a boolean and shows <code>documentAttached / uploadDocument</code>; payload 2459.js:340 <code>id_doc_uploaded: ge</code>; 631.js:4516 — <code>id_doc_uploaded: !!t.id_doc_uploaded</code> (stored, not required); 631.js:4468-4469 only checks that the document number has <code>>= 6</code> characters.	KYC for people who will collect money from players rests on a typed number and a tapped button; approvals are made on unverifiable data.	Implement an actual document capture (image picker / camera) stored as a reference, make it mandatory server-side, and have the admin detail screen display it.
F-ORG2-1 High	organizer/series/[id]	Financial control	Cancelling a series cancels all upcoming occurrences without refunding paid seats The single-match cancel path refunds every paid/pending seat and cancels the court booking; the series cancel path silently drops those steps. Paid players on every future occurrence lose their seat and keep a <code>paid</code> payment record with no refund and no <code>refund_issued</code> notification. Evidence: 631.js:5756-5806 — <code>mockCancelSeries</code> loops for <code>(const t of Kn(e))</code>, marks each game <code>status: "cancelled"</code> and each booking <code>e.status = "cancelled"</code> (5790-5797) but never calls <code>Br(...)</code> (the seat refund routine used by <code>mockCancelMatch</code> at 631.js:3622 for <code>(const e of o) await Br(n, e, !0);</code>). No <code>Mr</code> (court-booking cancel) call either.	Direct financial loss for players on paid series; venue court bookings stay live and billed; support disputes.	Reuse <code>mockCancelMatch</code> 's body (or extract a shared <code>cancelGameInternal</code>) inside the series loop so each occurrence releases holds, refunds paid seats (<code>Br(game, userId, true)</code>), cancels the court booking and writes a per-game <code>match.cancelled</code> audit entry.
F-ORG2-8 High	organizer/match/[id]	Authorization	Organizers can sanction any user in the app, not only participants of their own matches; game_id is unverified Nothing checks that <code>t</code> had a booking in a game organized by <code>e</code>, nor that <code>a.game_id</code> belongs to <code>e</code>. Any approved organizer can issue active warnings (which feed <code>computeStanding</code>, 631.js:12085) against arbitrary users, attributing them to arbitrary matches. The UI restricts the flag button to confirmed participants (2466.js:801-810) but that is client-side only. Evidence: 631.js:12100-12104 — <code>const i = ro(e) ? "admin" : "organizer"</code>; <code>if (("organizer" === i && (await on(e), oa(e, t)), e === t)) throw ... oa</code> (631.js:751-753) is only <code>ra(e, aa(t))</code>, i.e. an audience (gender-world) check. <code>game_id: a.game_id ?? null</code> (12118) is stored verbatim. Warnings become <code>status: "active"</code> immediately (12122, <code>needsAdminApproval</code> only for ban/suspension).	Harassment / reputation damage; conduct standing manipulated by unrelated organizers; audit trail records a misleading <code>game_id</code> .	In <code>mockIssueSanction</code> for organizer issuers, require <code>a.game_id</code> , load the game, <code>zi(game, e)</code> , and verify a non-cancelled booking for <code>t</code> in that game; reject otherwise.
F-XC-10 High	Platform (deployment)	Availability	Deployment configuration references files that do not exist Railway will fail at <code>npm install</code> (no <code>package.json</code>) and at <code>node build.mjs</code>. The TypeScript helpers <code>rpc.ts</code> and <code>auth/jwt.ts</code> are not imported by anything and depend on <code>jsonwebtoken</code>, which is not declared anywhere. <code>JWT_SECRET</code> is documented in <code>.env.example</code> but no code reads it in the shipped bundle. Evidence: <code>railway.json</code>:5-9 — <code>"buildCommand": "npm install && node build.mjs"</code>, <code>"startCommand": "node dist/host.mjs"</code>; <code>nixpacks.toml</code>:1-10 — same; repository root contains no <code>package.json</code>, <code>build.mjs</code>, or <code>dist/</code>	The site cannot be deployed from this repository as-is; the JWT configuration gives a false sense that server-side auth exists.	Either commit the Expo source project (with <code>package.json</code> , <code>app.json</code> , <code>app/ routes</code>) and a real <code>build.mjs/host.mjs</code> , or switch Railway to a static-site service that serves <code>index.html</code> . Delete or wire up <code>rpc.ts</code> and <code>auth/jwt.ts</code> .
F-XC-11 High	Platform (maintainability)	Availability	Source code is not in the repository; only the compiled bundle is The repository holds two identical 11.6 MB compiled HTML files and no source. None of the findings in this report can be fixed in this repo; they must be fixed in the original Expo project and re-exported. Evidence: git history — commits "Add files via upload" for <code>index.html</code> and <code>playora-standalone.html</code>; no <code>app/</code>, <code>src/</code>, <code>package.json</code>, <code>tsconfig</code> or tests; 631.js is a 16.5k-line minified module	No code review, no diffs, no CI, no tests, no way to hot-fix. Every change is a full re-upload.	Commit the Expo project source, add a CI pipeline (typecheck, lint, jest, Playwright smoke test of admin/organizer flows), and generate the single-file bundle as a build artifact rather than a committed file. Remove the duplicate <code>playora-standalone.html</code> .

ID / Severity	Interface	Risk Area	Identified Gap (with evidence)	Potential Impact	Recommended Remediation
F-XC-2 High	Platform (all interfaces)	Availability	Content-Security-Policy blocks all network calls, so the remote backend can never be used from the deployed page The page ships a CSP with <code>connect-src 'none'</code> . The backend health probe therefore always fails and the app silently locks itself into mock mode, even if a server were deployed at <code>rush-x.xyz</code> . The remote sign-in/OTP/reset paths in <code>913.js</code> are unreachable dead code in this build. Evidence: <code>index.html:5</code> — <code>connect-src 'none'</code> ; <code>673.js:23-33</code> — <code>fetch(`\${t}/health`) ... catch { return !1; }</code> ; <code>673.js:40-49</code> — falls back to <code>"mock"</code> when the health probe fails	Deploying a backend will have no effect until the HTML is rebuilt; developers may believe the server is being used when it is not.	When a backend exists, set <code>connect-src 'self' https://api.rush-x.xyz</code> (and remove the localhost default in <code>673.js:16</code>). Add a visible <code>"backend: mock/remote"</code> indicator in admin screens so the mode is never ambiguous.
F-XC-4 High	Sign-in (affects admin/organizer login)	Privacy	Third-party authentication endpoint hard-coded in the client A Neon Auth project URL is embedded in the bundle. Credentials are posted to it first, any failure is swallowed, and the code then retries against the configured backend. The response's <code>user.id</code> is accepted without checking it against the profile store, and no role is verified. Evidence: <code>913.js:68-77</code> — <code>fetch("https://ep-fancy-pond-zagvofc3.neonauth.c-2.eu-west-2.aws.neon.tech/neondb/auth/sign-in/email", { body: JSON.stringify({ email: e, password: t }) }) ... catch {}</code> then <code>h("/auth/signin", ...)</code>	Discloses infrastructure identifiers; creates two competing identity providers with different user ids; silent failures make login errors undiagnosable.	Remove the hard-coded URL; route all auth through the single configured API origin; surface the real error instead of <code>catch {}</code> .
F-XC-6 High	admin/* (all admin screens)	UX dead end	No legitimate path to an admin account in the deployed build (admin interface is a dead end) Admin role is granted only to emails listed in a build-time environment variable, which is empty in this bundle. The seeded "Platform Admin" profiles have no credentials, so nobody can sign in as them. The ADMIN section of the profile tab, and all eleven admin screens behind it, are therefore unreachable through the UI. Evidence: <code>bundle</code> — <code>process.env.PLAYORA_ADMIN_EMAILS??''</code> (inlined empty at build time); <code>631.js:487-491</code> — <code>le = new Set((process.env.PLAYORA_ADMIN_EMAILS ?? "").split(","))</code> ; <code>631.js:2062-2075</code> — promotion to admin only for emails in <code>le</code> ; <code>631.js:4700-4746</code> — seeded admin profiles have no matching entry in <code>Qt.users</code> (the only <code>Qt.users.push</code> is sign-up at <code>631.js:2240</code>); <code>1805.js:752</code> — admin section rendered only when <code>"admin" === M?.role "analyst" === M?.role</code>	The entire admin interface cannot be exercised or QA-tested from the deployed site; the only way in is editing <code>localStorage</code> , which is also the attack path in F-XC-1.	Provision admin accounts server-side with a role table and an audited "grant role" action; remove build-time email lists.
F-XC-8 High	Platform (every mutation)	Data integrity	Persistence failures are swallowed; the UI reports success even when nothing was saved Every write serialises the entire collection (all games, all payments, all notifications) to a single <code>localStorage</code> key. When the ~5 MB quota is exceeded, or storage is blocked (private mode, Safari ITP), the write fails, the error is logged to the console only, and the calling screen proceeds as if the mutation succeeded. On the next reload the change is gone. Evidence: <code>631.js:1727-1740</code> — <code>Za = async (e, t) => { ... try { await l.default.setItem(e, JSON.stringify(t)); } catch (t) { noteWriteFailure(e, t); } ... }</code> ; <code>643.js:53-58</code> — <code>noteWriteFailure</code> only increments a counter and <code>console.errors</code> ; <code>persistenceHealth</code> (<code>643.js:69-74</code>) is not referenced by any screen	Silent data loss of approvals, scores, sanctions and payments; admins and organizers cannot tell that their action did not persist.	Make <code>Za</code> rethrow (or return a failure the caller must handle) and show a blocking error toast; surface <code>persistenceHealth()</code> in the admin insights screen; move to server-side storage.
F-XC-9 High	Sign-in / sign-up (all roles)	Privacy	Password hashes, sessions and lockout counters live in localStorage Credentials are stored in the browser with a fast, non-memory-hard hash. The "secure" store is a naming prefix only. Session tokens never expire and are not validated by any server. Brute-force lockout is stored client-side and can be reset by the attacker. Evidence: <code>631.js:2240</code> — <code>Qt.users.push({ id, email, salt, passwordHash, failedAttempts, lockedUntil })</code> persisted under <code>playora.mock.users.v1</code> ; <code>632.js:66-70</code> — fallback hash is 12,000 iterations of SHA-256 (script unavailable in browser); <code>637.js:6-13</code> — <code>secureSet</code> is a plain <code>setItem("secure." + key)</code> ; <code>913.js:59-63</code> — session written to <code>localStorage["secure.playora_session"]</code> ; <code>631.js:2270-2280</code> — lockout after 5 failures stored in the same editable record	Any XSS, malicious extension, or shared computer exposes every registered user's email and hash; sessions cannot be revoked.	Authenticate server-side (argon2id/bcrypt), issue short-lived JWTs with refresh and revocation (the <code>repo's auth/jwt.ts</code> is a starting point but is unused), and store tokens in <code>HttpOnly</code> cookies or at least memory + secure storage on native.
F-ADM1-11 Medium	admin/organizers, admin/players, admin/venues, admin/conduct	Ambiguity	Access-denied gate is labelled "Verification required" and is triggered by any fetch error The gate text was written for the organizer verification wall and is reused for admin authorization; and because the catch is unconditional, a partition refusal, a storage read failure or any thrown error renders the same lock screen. The error object is discarded so nothing is reported. Evidence: <code>1832.js:27-28</code> <code>catch { F(!0); }</code> then <code>1832.js:46-50</code> <code>EmptyState({ icon: "lock-closed-outline", title: W("organizerGateTitle"), body: W("adminPanel") })</code> ; identical pattern at <code>1833.js:22-23/72-76</code> , <code>1834.js:22-24/56-60</code> , <code>1826.js:19-21/53-57</code> ; <code>909.js:1201</code> <code>organizerGateTitle: "Verification required"</code> , <code>909.js:1158</code> <code>adminPanel: "Admin"</code> .	Real admins hit "Verification required / Admin" on transient failures and have no retry; non-admins get an inaccurate explanation.	Distinguish <code>E_ADMINISTRATOR_AUTHORIZATION_REQUIRED</code> from other errors, show <code>storeErrorText(e.message)</code> and a Retry button, and add an admin-specific title key.

ID / Severity	Interface	Risk Area	Identified Gap (with evidence)	Potential Impact	Recommended Remediation
F-ADM1-12 Medium	admin/players	Audit trail	Rating +/- steppers have no error handling, no loading state, no confirmation and a hard-coded audit note Every tap mutates a player's official rating (persisted at 631.js:5443) with no reason captured — the "note" is always the literal string "admin panel" — and no confirmation. If the mock throws (auth, storage) the rejection is unhandled and the user sees nothing. Rapid taps race with the <code>Y()</code> refresh (each tap uses the possibly stale <code>t.rating</code> from the last render). Evidence: 1833.js:31-36 — <code>const Z = async (t, r) => { ... await adminAdjustRating(e.id, t.user_id, t.sport, t.rating + r * o, "admin panel"), await Y(); } (no try/catch, no busy flag); 1833.js:233 and 254 <code>onPress: () => Z(e, -1) / Z(e, 1);</code> 631.js:5444-5448 — audit meta <code>note: sanitizeText(n ?? "", 120) "manual adjustment"</code>. </code>	Silent, unexplained rating manipulation; audit entries that all read "admin panel!"; unhandled promise rejections in the console.	Route Z through a try/catch with <code>Alert</code> , disable steppers while pending, require a reason (prompt once per player/session), and pass it as the note.
F-ADM1-13 Medium	admin/players	Authorization	mockAdminAdjustRating has no partition check and will create a skill profile for any id The API accepts any <code>user_id/sport</code> and will write a rating for a non-existent user or a user in the other world, bypassing the partition policy applied to every other admin surface. Evidence: 631.js:5437-5449 — <code>(await ei(), await sn(e)); const r = await Dn(t, a), o = r.rating; r.rating = clampRating(a, i) ...</code> — no <code>_n(e, un(t), ...)</code> call, unlike <code>mockAdminCorrectAudience</code> at 631.js:4426; 631.js:5127-5150 — <code>Dn</code> pushes a brand-new profile (<code>Qt.skillProfiles.push(a)</code>) when none exists and never checks that <code>Qt.profiles</code> contains <code>t</code> .	Orphan skill rows; inconsistent enforcement of the world partition; <code>Dn</code> side-effect on a read path when the id is mistyped.	Add <code>_n(e, un(t), "admin.adjust_rating")</code> , verify the profile exists, and use a non-creating lookup for the adjustment.
F-ADM1-14 Medium	admin/players	Audit trail	Every intel refresh (including after each +/- tap) writes an "intel.accessed" audit row into a 500-entry capped log The access log is correct in principle, but combined with the fixed-size ring buffer, routine refreshes evict older privileged-action records (approvals, bans, commission changes) from the only audit store. Evidence: 631.js:5433 — <code>await logAdminAudit("intel.accessed", e, null, { scope: "full", rows: t.length });</code> 1833.js:35 <code>await Y()</code> after every adjustment and 1833.js:382 after audience correction; 643.js:25 <code>o = 500</code> and 643.js:43 <code>u.slice(0, o)</code> .	Loss of the audit trail for real decisions after ~500 events; noisy log makes review impractical.	Log intel access once per session/screen mount (not per refresh), and raise or remove the cap for admin events (or ship them via <code>appendAudit</code> , 643.js:48-49, to a server store).
F-ADM1-16 Medium	admin/players	Data integrity	Audience correction silently deletes all of the player's follow relationships A privacy-motivated side effect (severing social links across worlds) is undisclosed to the admin, unrecorded in the audit meta and irreversible. Evidence: 631.js:4429-4430 — <code>Qt.follows = Qt.follows.filter((e) => e.follower_id !== t && e.followee_id !== t), await Za(Ne, Qt.follows);</code> modal copy 909.js:706-707 "This is the only way the fork choice can change. It is recorded with your name." (no mention of follows); audit meta 631.js:4431-4436 does not record the number of follows removed.	Unexpected loss of user data; support cannot explain why follows disappeared.	State the side effect in <code>adminCorrectWorldBody</code> log <code>follows_removed</code> in the audit meta, and consider soft-deleting.
F-ADM1-2 Medium	admin/application/[id]	Audit trail	Reject / Suspend decisions do not require a reason; a canned reason is substituted silently The reason field (1823.js:257-266) is optional and the backend back-fills a generic string, which is then stored as <code>rejection_reason</code> , sent to the applicant (wn, 631.js:4599/4617) and written to the audit log (631.js:4598/4616). Additionally the input allows 500 chars (1823.js:264) but reject/suspend truncate the reason to 200 (631.js:4590/4609) with no warning. Evidence: 1823.js:32 — <code>reviewApplication(c.id, e, s, { reason: G, notes: G })</code> (G may be ""); 631.js:4590 — <code>sanitizeText(i?.reason ?? "", 200) "Did not meet verification requirements."</code> ; 631.js:4609 — <code> "Policy violation."</code> ; 631.js:4601 — <code>request_info</code> defaults to "Please provide additional information.".	Adverse decisions (rejection with a 30-day reapply lock, 631.js:4596; suspension) can be issued with no documented rationale; the audit record shows a boilerplate reason indistinguishable from a real one.	Require a non-empty reason for reject/suspend in both the screen (disable buttons while <code>!G.trim()</code>) and the mock (<code>throw new Error("E_A_REASON_IS_REQUIRED")</code>), as <code>mockAdminCorrectAudience</code> already does at 631.js:4420). Align <code>maxLength</code> with the server cap.
F-ADM1-20 Medium	admin/venues	Audit trail	Venue reject/suspend capture no reason, have no confirmation, and the owner cannot tell suspension from rejection The backend supports a <code>review_note</code> but the screen never collects one; suspending an approved venue (which stops all bookings via <code>yr</code> , 631.js:6413-6417) is a one-tap action; the owner receives <code>approved: false</code> for both reject and suspend. Evidence: 1834.js:167, 175, 183 — <code>reviewVenue(e.id, t.id, "approve" "reject" "suspend")</code> (no 4th argument, no confirm); 631.js:6712 — <code>n.review_note = i ? sanitizeText(i, 300) : null;</code> 631.js:6714-6723 — owner notification carries only <code>approved: "approve" === a;</code> 631.js:6724-6733 — audit meta is <code>{ venue: t.slice(-6) }</code> with no reason.	Undocumented revenue-affecting decisions; venue owners cannot understand or contest the outcome.	Add a reason prompt (required for reject/suspend), pass it as the 4th arg, include it in the audit meta, and add a <code>decision</code> field to the notification.

ID / Severity	Interface	Risk Area	Identified Gap (with evidence)	Potential Impact	Recommended Remediation
F-ADM1-21 Medium	admin/venues	Financial control	Commission edit: malformed input silently becomes 0, no previous value is logged, no confirmation A typo sets a venue's commission to 0 % with no warning; the audit cannot show what it was before; fixed commissions can exceed the court price. Evidence: 1834.js:255 — <code>setVenueCommission(e.id, t.id, J, Number(U) 0)</code> ; 1834.js:245 input filter allows multiple dots (<code>"1.2.3"</code> → <code>NaN</code> → <code>0</code>); 631.js:6741 — fixed type has no upper bound (<code>Math.max(0, i)</code>); 631.js:6745-6748 — audit meta <code>{ venue, commission: "type:value" }</code> records only the new value; no confirmation step on the screen.	Revenue loss until noticed; weak evidence for reconciliation.	Validate numerically (reject <code>NaN</code> , require <code>0 < value</code> , cap fixed at a sane maximum), confirm with a before → after summary, and log <code>from/to</code> in the audit meta.
F-ADM1-23 Medium	admin/venues	Financial control	Financial tiles sum all settlements including "pending" ones, so "Payouts" overstates money paid out Pending and settled amounts are mixed; the tile presented as payouts is really "net owed + paid". Refunds (631.js:7962-7964) are summed separately from payments, so gross is not net of refunds. Whether a "settled" state exists needs confirmation, but the code path never distinguishes. Evidence: 631.js:7957-7961 — <code>grossKwd: roundKwd(t.reduce((e, t) => e + t.gross_kwd, 0))</code> ... <code>payoutsKwd: roundKwd(t.reduce((e, t) => e + t.net_to_venue_kwd, 0))</code> over <code>Qt.settlements</code> with no status filter; 631.js:6858-6868 — settlements are created with <code>status: "pending"</code> , <code>settled_at: null</code> ; 631.js:7030 shows pending settlements can later be removed on cancellation. Screen labels the tile <code>payoutsLabel</code> (1834.js:93-97).	Misleading platform financial summary for an admin making commission decisions.	Split into pending vs settled (<code>settled_at !== null</code>), and label gross as "before refunds" or subtract refunds.
F-ADM1-24 Medium	admin/conduct	Audit trail	Sanction Approve / Reject / Overturn capture no reason and have no confirmation Approving a ban (which the app treats as a serious, appealable action — organizer-issued bans require admin approval, 656.js <code>needsAdminApproval</code>) or overturning an active sanction is a single tap with no rationale stored anywhere. Evidence: 1826.js:183, 190, 200 — <code>reviewSanction(e.id, t.id, "approve" "reject" "overturn")</code> (no 4th arg, no confirm); 631.js:12172 — <code>n.review_note = i ? sanitizeText(i, 300) : null</code> ; 631.js:12187 — audit meta <code>{ decision: a, type: n.type }</code> (no note); the player notification (631.js:12176-12185) carries only <code>upheld</code> .	Bans upheld or lifted without documented justification; appeals cannot be reviewed.	Prompt for a note (required for approve-ban and overturn), pass it through, include it in the audit meta and notification.
F-ADM1-29 Medium	admin/conduct	Segregation of duties	Admin-issued bans/suspensions skip the approval queue entirely Only organizer-issued bans need a second pair of eyes; an admin's own ban is active immediately and never appears as "pending" in this queue, so the review workflow shown on this screen does not cover the most powerful issuer. Evidence: 656.js (raw bundle) — <code>needsAdminApproval=(n,o)=>'organizer'===o&&('ban'===n 'suspension'===n)</code> ; 631.js:12106-12123 — <code>const i = ro(e) ? "admin" : "organizer"; ... status: r ? "pending_approval" : "active"</code> .	A single admin can permanently ban any user with no review; the "Pending review" counter understates the true number of unreviewed bans.	Route admin-issued bans through <code>pending_approval</code> too and forbid the issuer from approving their own (<code>n.issued_by === e</code> check in <code>mockReviewSanction</code>).
F-ADM1-3 Medium	admin/application/[id]	Data integrity	No confirmation dialog before Reject or Suspend organizer Destructive, hard-to-reverse decisions execute on a single tap with no confirm step, while both buttons sit next to Approve. Evidence: 1823.js:293-311 — <code>onPress: () => K("reject")</code> / <code>onPress: () => K("suspend")</code> call the mutation directly; 631.js:4596 sets a 30-day <code>reapply_after</code> , 631.js:4597/4615 demote the user's role.	Accidental rejection locks the applicant out for 30 days; accidental suspension revokes organizer rights immediately.	Wrap reject/suspend in <code>Alert.alert</code> confirm (as done elsewhere in the app), showing the reason that will be sent.
F-ADM1-6 Medium	admin/application/[id]	UX dead end	Detail screen swallows all fetch errors into a bare "Error" view with no back button or cause Three very different outcomes (not admin, record in other partition, id not found) collapse into the word "Error", and the screen has no chrome to navigate back (the header at 1823.js:61-78 is only rendered in the success branch). The screen also has no client-side role gate unlike 1832.js:40. Evidence: 1823.js:18-21 — <code>W(await fetchApplicationDetail(c.id, e));</code> <code>} catch { W(null); }</code> ; 1823.js:48-55 — renders only <code>V("error")</code> text inside a <code>SafeAreaView</code> with no header/back control; 631.js:4304 throws <code>E_ADMINISTRATOR_AUTHORIZATION_REQUIRED</code> , 631.js:4338 throws <code>E_THAT_RECORD_IS_IN_THE_OTHER_WORLD</code> , 631.js:4570 returns <code>null</code> when not found.	Deep-linked or stale ids leave the admin stuck on a blank error page; partition refusals (which are audited, 631.js:4337) look like bugs.	Keep the header/back button in the error branch, surface <code>storeErrorText(e.message)</code> (already imported as <code>S</code> , 1823.js:37), and add the same <code>"admin" !== profile?.role</code> gate used at 1832.js:40.
F-ADM1-7 Medium	admin/application/[id]	Data integrity	Rejecting/suspending (or approving) an application overwrites non-admin roles such as "analyst" The role write is unconditional for every non-admin role. An analyst who applies to organize and is rejected becomes a plain "user"; approved, they become "organizer" and lose analyst access (1805.js:752 gates the BI entry on <code>"analyst" === role</code>). Evidence: 631.js:4597 and 4615 — <code>await mn(n.user_id, "user")</code> ; 631.js:4586 — <code>await mn(n.user_id, "organizer")</code> ; 631.js:4437-4442 — <code>mn</code> only guards <code>"admin" !== Qt.profiles[a].role</code> ; 631.js:4746 seeds a profile with <code>role: "analyst"</code> ; 631.js:10658 <code>Ns = new Set(["admin", "analyst"])</code> shows analyst is a privileged role.	Silent privilege loss (or loss of BI access) with no audit entry describing the role change (the audit at 631.js:4598 only logs the reason).	Make <code>mn</code> preserve any privileged role (<code>admin</code> , <code>analyst</code>) or model organizer status as a separate flag instead of overwriting <code>role</code> ; log role transitions.

ID / Severity	Interface	Risk Area	Identified Gap (with evidence)	Potential Impact	Recommended Remediation
F-ADM2-13 Medium	admin/optimizer	Data integrity	Optimizer feature-importance card is stale after saving weights After a stepper tap the weight row changes but the importance bars (derived server-side from the same weights) keep the previous values until the screen is re-focused. Evidence: 1831.js:29-31 — L only updates F (weights) via K(...); 1831.js:113 renders v.feature_importance from the dashboard state v, which is refreshed only in D (1831.js:14-23) on focus; 631.js:12594-12598 derives feature_importance from yd().	Admin sees contradictory numbers on one screen and may over-adjust.	Re-fetch the dashboard after setOptimizerWeights resolves, or compute importance client-side from F.
F-ADM2-14 Medium	admin/insights	Ambiguity	Export silently falls back to the Executive report for the Sports and Health tabs With the Sports or Health tab selected, the download button produces playora-executive-<date>.csv and the success alert shows that filename; nothing tells the user the selected tab was not exported. Evidence: 1830.js:68-78 — ["executive", "financial", "users", "matches", "venues", "organizers", "geographic"].includes(E) ? E : "executive"; tabs list 1830.js:214-224 includes sports and health; 631.js:11000-11050 has no branch for those sections (the final else is geographic).	Wrong report delivered without notice.	Either add sports/health branches server-side or hide/disable the export button on those tabs with a hint.
F-ADM2-15 Medium	admin/insights	Data integrity	CSV export has no formula-injection guard on user-supplied labels Values beginning with =, +, -, @ are written verbatim. An organizer whose display name is =HYPERLINK(...) or =cmd ... lands in an admin's spreadsheet. Evidence: index.html toCsv=function(e,t){const n=e=>{const t=String(e);return["",\n]/.test(t)?`\${t.replace("/g","")}`:t}... (called at 631.js:11060); exported rows include organizer names e.name (631.js:11043), venue labels (631.js:11039), area names (631.js:11051) which originate from user/venue-editable fields.	Classic CSV injection against the admin who opens the export in Excel/Sheets.	Prefix cells starting with = + - @ \t \r with a single quote (or a space) in toCsv.
F-ADM2-17 Medium	admin/insights	Availability	A single failed dashboard fetch permanently locks the Insights screen Any transient error (e.g. a persistence read failure during Gs, which calls five loaders at 631.js:10663) flips the screen into the lock state until the component unmounts; re-focus re-runs ee but the successful result is hidden because Q stays true. Evidence: 1830.js:23-27 — catch { X(!0) } sets Q; 1830.js:41 — if (Q !V) renders the lock screen; Q is never reset to false anywhere in the module, and the lock screen (1830.js:42-56) renders no filter chips or retry.	Admin sees "Verification required" and has to navigate away and back.	Reset X(false) at the start of ee and add a retry control.
F-ADM2-20 Medium	admin/feed	Data integrity	Feed ranking weights self-modify from any user's clicks with no admin control or audit The platform-wide feed weights shown on this screen are mutated by ordinary user interactions, never logged via logAdminAudit, and the admin screen offers no way to freeze, reset or override them. Evidence: 631.js:15181-15206 — mockLogFeedSignal (no role check) ... 631.js:15203 Qt.feedWeights = o calls nudgeWeights(Ml(), t, r) and persists Qt.feedWeights = o on every join/click/cancel; 1828.js:100-141 renders \$.weights read-only; no setFeedWeights exists in 671.js.	A user (or bot) clicking/cancelling repeatedly can steer global ranking; admin can only watch.	Add a mockSetFeedWeights/reset action guarded by sn with audit, bound the nudge per user/day, and log weight drift.
F-ADM2-3 Medium	admin/funnel	Data integrity	Any client can inject arbitrary funnel events and flood the audit log The event name is free text from the caller with no whitelist, no rate limit and no auth beyond a session id. Combined with F-ADM2-2 (500-entry cap), 500 calls evict every other audit record. Evidence: 631.js:15163-15166 — mockLogFunnel = async (e, t) => { await ei(), await logAudit("funnel.event", actorRef(e), { e: sanitizeText(t, 60) }) }; 631.js:15170-15176 — stats group by e.meta?.e ?? "unknown".	The funnel screen can be polluted with fabricated event rows; the admin audit trail can be wiped by a single user.	Validate t against a fixed enum of funnel step names; rate-limit per user; store funnel counters outside the audit log.
F-ADM2-4 Medium	admin/demand, admin/optimizer, admin/concierge	Validation	Backend weight setters accept NaN and arbitrary keys (mass assignment) Math.max(0, Math.min(1, NaN)) is NaN, so a non-numeric or missing value is persisted as NaN. Unknown keys in t are merged and persisted as weights. There is no check that a key is a known weight, that a value is a finite number, or (optimizer) that the success weights are non-degenerate. Evidence: 631.js:12352-12358 — Object.assign({}, fd(), t, ...) then for (const e of Object.keys(a)) ... a[e] = Math.max(0, Math.min(1, Math.round(100 * a[e]) / 100)); 631.js:12556-12558 same for optimizer; 631.js:11698-11706 — concierge clamps with Math.max/min/round only.	A malformed payload (or a future screen bug) poisons Qt.demandWeights/Qt.optimizerWeights; downstream normalizeImportance renders NaN% and predictions become NaN (631.js:12268, 12399 consume these). The UI stepper then shows NaN with no way to recover except reseeding.	Whitelist keys against DEFAULT_*_WEIGHTS, reject non-finite numbers with a store error, and ignore extra keys.

ID / Severity	Interface	Risk Area	Identified Gap (with evidence)	Potential Impact	Recommended Remediation
F-ADM2-5 Medium	admin/concierge, admin/demand, admin/optimizer	Bug/crash	Stepper saves are unguarded — unhandled rejection, no feedback, lost updates on rapid taps No try/catch, no in-flight flag, no toast. A failed save (e.g. <code>E_ADMINISTRATOR_AUTHORIZATION_REQUIRED</code> after a role change, or a persistence failure) surfaces only as an unhandled promise rejection. Two quick taps both send <code>S+step</code> from the same stale closure, so the second overwrites the first (net +1 step instead of +2), and out-of-order responses can leave the display at a value that is not what was persisted. Evidence: 1825.js:29-33 — <code>const L = async (l) => { if (!t) return; const n = await setConciergeRules(t.id, l); R(n); }</code> ; 1827.js:29-31 — <code>t && I(await setDemandWeights(t.id, l))</code> ; 1831.js:29-31 — <code>t && K(await setOptimizerWeights(t.id, l))</code> ; <code>onDec/onInc</code> compute from the rendered value, e.g. 1827.js:129 <code>H({ [t.key]: Math.max(t.min, S[t.key] - t.step) })</code> .	Admin believes a weight changed when it did not, or sees a value different from the stored one.	Wrap in try/catch with an error alert, disable the stepper while a save is pending (or serialise via a ref), and compute the next value from the latest server response.
F-ADM2-6 Medium	admin/awards, admin/funnel	Ambiguity	Errors on awards/funnel are rendered as "all clear" empty states Authorization failures, persistence read failures or any thrown error are indistinguishable from a genuinely clean result. For the fraud screen this is a false negative on a control report. Evidence: 1824.js:15-16 — <code>catch { A([]); }</code> then 1824.js:59-63 renders <code>noFraud</code> ("No suspicious voting detected."); 1829.js:17 — <code>.catch(() => z([]))</code> then 1829.js:54-58 renders <code>funnelEmpty</code> ("No funnel events yet").	An analyst (or an admin whose role was revoked mid-session) reads "No suspicious voting detected" and signs off; an outage reads as "no fraud".	Keep a separate error state (as 1828 does) and render an error/retry component; never coerce a rejection into <code>[]</code> .
F-ORG1-1 Medium	organizer/new	UX dead end	organizer/new is an orphan third match-creation wizard with its own (weaker) rules A complete 7-step wizard (sport/when/where/format/formation/price/review) with draft autosave, discard confirmation and a success screen exists but nothing in the app links to it. It is the only consumer of <code>saveMatchDraft/fetchMatchDraft/discardMatchDraft</code> (671.js:465-467, grep shows only 2468.js uses them), so the whole draft subsystem is effectively dead. On web the route is still reachable by URL, so the divergent behaviour in F-ORG1-2/3/4 is live. Evidence: 1809.js:108 — <code>name: "organizer/new"</code> ; <code>routes.txt:58</code> — <code>./organizer/new.tsx 2468</code> ; grep of all <code>pretty*.js</code> for <code>"/organizer/new"</code> returns only the stack registration; 2468.js:48-68 — <code>draft restore</code> ; 2468.js:728-757 — <code>createMatch(... formation_key, spot_assignment ...)</code> .	Maintenance burden and two sets of business rules; a user who lands on the URL gets a flow that hard-codes venue coordinates and drops fields. Drafts saved here are never surfaced elsewhere.	Either delete <code>organizer/new</code> (and the draft API) or make it the single canonical flow; do not ship both. If kept, add it to navigation and align validation with <code>create</code> .
F-ORG1-12 Medium	organizer/apply	UX dead end	Suspended organizers never see the suspension reason and have no way forward A suspended organizer is redirected here from the dashboard (2465.js:22) and from <code>create</code> (2461.js:151) and sees a generic card with no reason and no CTA other than Back. Evidence: 631.js:4609-4613 — <code>suspension</code> stores the reason in <code>rejection_reason</code> ; 2459.js:460-475 — the reason card is rendered only when <code>"rejected" === e.status</code> ; 2459.js:417-422 — the <code>suspended</code> card has title/body only; 2459.js:493-501 — action buttons exist only for <code>approved/rejected</code> ; error text for the backend guard says "contact" (674.js:249 <code>seYourOrganizerAccessIsSuspendedContact</code>) but no <code>contact/appeal</code> link is rendered although a <code>/contact</code> route exists (1809.js).	No due process for suspensions; support load.	Show <code>rejection_reason</code> for <code>suspended</code> , add an "appeal / contact support" button (route <code>/contact</code>), and record appeals.
F-ORG1-13 Medium	organizer/apply (backend <code>mockSubmitOrganizerApplication</code>)	Audit trail	Re-application overwrites the previous application record; velocity control is dead code One record per user is kept, so the prior rejection (reason, reviewer, dates, risk flags) is destroyed on resubmission and only the audit log line (4531) remains. Because there is never more than one record per user, the <code>">= 3 submissions in 24 h"</code> velocity flag can never trigger. Evidence: 631.js:4501-4529 — new record built with <code>id: d?.id ?? ea()</code> , <code>rejection_reason: null</code> , <code>admin_notes: null</code> , <code>reviewed_by: null</code> , <code>reviewed_at: null</code> , then <code>g >= 0 ? (Qt.applications[g] = f) : Qt.applications.push(f)</code> ; 631.js:4483-4485 — <code>Qt.applications.filter((t) => t.user_id === e && ...< 864e5).length >= 3 && u.add("velocity")</code> .	Reviewers cannot see that an applicant was previously rejected or why; repeat submissions cannot be detected as intended.	Keep applications immutable and append a new version (or store a <code>history</code> array); compute velocity across the history; surface previous decisions in the admin detail screen.
F-ORG1-16 Medium	organizer/index, organizer/create	Authorization	Organizer read endpoints are keyed only on the target organizer id (no caller check) The API contract has no notion of "caller"; any client can read any organizer's private invite codes, ratings, pending bookings and court bookings by passing an id. The brief notes the mock is client-side, but these signatures will carry over to a real backend unchanged. Evidence: 631.js:3962 <code>mockGetOrganizerMatches = async (e)</code> returns full game objects via <code>Ii</code> (2615-2632) including <code>invite_code</code> of private matches (set at 3414); 3984 <code>mockGetOrganizerStats(e)</code> ; 4052 <code>mockGetOrganizerRatings(e)</code> (includes <code>player_name</code>); 5835 <code>mockGetOrganizerSeries(e)</code> ; 5988 <code>mockGetOrganizerReferralStats(e)</code> ; 7096 <code>mockGetBooking = async (e, t)</code> ignores the second (caller) argument entirely and returns any court booking by id (<code>Pr(a)</code>), whereas <code>Wi</code> checks ownership only at publish time (3348).	Private-match invite codes and other organizers' operational data leak; when ported to a server these become IDOR endpoints.	Give every read endpoint a caller parameter and assert <code>caller === organizer</code> or admin; strip <code>invite_code</code> from list responses.

ID / Severity	Interface	Risk Area	Identified Gap (with evidence)	Potential Impact	Recommended Remediation
F-ORG1-17 Medium	organizer/create (recurring)	Validation	mockCreateSeries lacks the safeguards that mockCreateMatch has A series can be created starting in the past, with an end date before its start, or overlapping the organizer's own matches, and generates occurrences immediately (<code>jn(n)</code> 5672). Evidence: 631.js:5603-5612 validates title, end>start minutes, players 2..40, weekday/monthly pattern only. Missing compared with <code>Wi</code> : no past-date check (cf. 3329), no <code>end_date >= start_date</code> check (5662 stores <code>t.end_date ?? null</code> verbatim), no rate limit (cf. 3336-3342), no overlap/duplicate check (cf. 3383-3395), no venue-supports-sport check, <code>format/visibility/approval_mode</code> unvalidated (5643, 5650-5651). Client side 2461.js:1024-1034 also does not check date in past or end date.	Bulk creation of invalid/overlapping occurrences; unbounded spam of series (no rate limit) each materialising up to <code>horizon_weeks: 12</code> of games.	Reuse <code>Wi</code> 's validation block for the template and validate each generated occurrence; add a per-organizer series rate limit and end-date sanity check.
F-ORG1-19 Medium	organizer/create (custom venue), organizer/new	Segregation of duties	Any approved organizer can publish new venues to the global venue list without review Venue creation bypasses the venue application/approval process (<code>venue/apply</code> route exists) and is not audited or reviewable. Evidence: 631.js:3354-3377 and 5614-5633 — <code>custom_venue</code> becomes a full record in <code>Qt.venues</code> with <code>custom: !0</code> ; <code>fi()</code> (2447-2450) returns all <code>Qt.venues</code> to every caller; no audit line specific to venue creation (only <code>match.created</code> at 3457). 2461.js:646-654 offers "custom venue" as a first-class tab.	Duplicate/fake venues pollute search, maps and smart-schedule inputs; no owner attribution.	Mark organizer-created venues as private to that organizer (or pending admin review), log <code>venue.created</code> with the actor, and de-duplicate by name/coordinates.
F-ORG1-2 Medium	organizer/create, organizer/new, organizer/quick	Data integrity	Three creation flows (create, new, quick) apply different validation, defaults and post-create routing Same entity, three rule sets. A match's price cap, whether manual approval is possible, and what "quick" means depend on which entry point the organizer happened to use. Evidence: - Price: 2461.js:220 <code>Kt = "" !== gt.trim() ? Number(gt) 0 : pt</code> (no client max; backend clamps to 0..100 at 631.js:3332); 2468.js:94 <code>Number(Se) >= 0 && Number(Se) <= 50</code> ; 2470 never shows a price and lets the backend choose (631.js:3501-3503 <code>\$i = {football:{players:10,price:2.5},...}</code> , 3530). - Approval mode: 2461.js:992-1000 user picks auto/manual independently of visibility; 2468.js:742 <code>approval_mode: "private" === ke ? "manual" : "auto"</code> (derived, not selectable); 2470/631.js:3531-3532 always <code>visibility: "public"</code> , <code>approval_mode: "auto"</code> . - Formats/capacity: 2461.js:1175-1190 includes <code>padel_2</code> and <code>football_custom</code> (2..40 stepper, 873-874); 2468.js:898-909 has neither, capacity is fixed by format; 2470 capacity comes from <code>Ki</code> history or the sport default (631.js:3511). - Duration: 2461 free start/end minutes (446 auto-sets end = start+90); 2468.js:915 <code>_ = [60, 90, 120]</code> ; 2470 backend <code>s = i?.duration_minutes ?? 90</code> (631.js:3513). - Post-create: 2461.js:1112 and 2470.js:449 <code>replace('/organizer/match/:id')</code> ; 2468.js:991/1027 <code>replace('/game/:id')</code> (public game page) after a 4 s timer. - Naming: the create screen has an internal "quick" lane (2461.js:262 <code>laneQuick</code> , 1016 <code>quickPublish</code>) in addition to the separate <code>/organizer/quick</code> screen (2465.js:92 <code>quickMatchCta</code>).	Inconsistent data (e.g. a 70 KWD match from <code>create</code> , impossible from <code>new</code>), support confusion, and tests/controls written for one flow silently bypassed by another.	Centralise match-input validation (price ceiling, capacity/format table, approval rules) in one shared validator used by all client flows and by <code>Wi</code> ; reduce to one full flow and one quick flow with a single "quick" name.
F-ORG1-3 Medium	organizer/new	Data integrity	organizer/new hard-codes the coordinates of every custom venue The wizard has no location picker (unlike 2461.js:732 <code>LocationPicker</code>), so every venue created here is stored at the same fixed point in Kuwait City, and the "area" input is stored as the address. Evidence: 2468.js:748-753 — <code>custom_venue: { name: de.trim(), address: pe.trim(), lat: 29.32, lng: 47.99 }</code> ; 631.js:3357-3359 accepts any numeric lat/lng and 3361-3377 pushes a <code>custom: !0</code> venue into <code>Qt.venues</code> .	Distance/radius features, maps and smart-schedule scoring (which use venue lat/lng) are wrong for these venues; players are directed to the wrong place.	Require a real location (reuse <code>LocationPicker</code>) or reject custom venues in this flow; backend should reject obviously placeholder coordinates or flag <code>custom</code> venues for admin review.
F-ORG1-5 Medium	organizer/create (? court_booking_id=)	Bug/crash	Court-booking prefill in create sets the sport but not the format/capacity A padel or tennis court booking produces a match with <code>sport: "padel"</code> , <code>format: "football_5v5"</code> , <code>max_players: 10</code> , and an auto title like "Padel 5 v 5" (2461.js:158 <code>\$t = \"\${G(ne)} \${G(fmt_\${de})}\"</code>). In the default "quick" lane the format chips are hidden (2461.js:388-403 only under <code>"custom" === Nt</code>) so the organizer cannot even see or fix it. Evidence: 2461.js:195-212 — <code>(re(\"\${e.venue_name} . \${e.court_name}\\\" , ce(e.sport))</code> then only date/time/venue are set; compare the URL-param path 2461.js:187 which calls <code>Vt(U.sport)</code> ; <code>Vt</code> (2461.js:162-176) is what sets <code>de</code> (format), <code>pe</code> (players), <code>dt</code> (waitlist) and skill scale. Initial state: 2461.js:16-17 <code>de = "football_5v5"</code> , <code>pe = 10</code> .	Wrong capacity (10 players on a padel court), wrong format label, and downstream format-based logic (lineups, <code>Ia</code>) applied to the wrong sport. <code>Wi</code> does not validate sport/format consistency (631.js:3403-3404 stores both verbatim).	Call <code>Vt(e.sport)</code> in the booking effect (and guard <code>Vt</code> against unknown sports); validate on the backend that <code>format</code> belongs to <code>sport</code> .

ID / Severity	Interface	Risk Area	Identified Gap (with evidence)	Potential Impact	Recommended Remediation
F-ORG1-6 Medium	organizer/create	Data integrity	Optimizer in create is computed once from initial inputs; quick-lane publish silently uses those stale values Changing venue, date, time, price or capacity after mount never re-runs the optimizer. In the default quick lane the organizer sees date/time/venue inputs but the published price, capacity, skill and waitlist come from <code>Mt.optimal</code>, which was derived for a different venue/slot (the "optimal" slot, not the one chosen) and is never re-evaluated. The suggested price passed from smart-schedule / concierge (2461.js:193 <code>U.price && ht(U.price)</code>) is also overridden by <code>o?.price_kwd</code> in this lane. The format <code>de</code> is not adjusted to <code>o.max_players</code>, so e.g. <code>football_5v5</code> with 14 players is possible. Evidence: 2461.js:71-102 — the optimization effect passes <code>venue_id, starts_at, price_kwd, max_players, ...</code> but its dependency list is <code>[e, ne, X, Dt]</code> (sport, booking id, dismissed flag only). 2461.js:1085-1098 — <code>o = "quick" === Mt && !X && Mt ? Mt.optimal : null</code> and <code>skill_level: o?.skill_level ?? lt, max_players: o?.max_players ?? pe, waitlist_capacity: o ? o.waitlist_cap : dt, price_kwd: o?.price_kwd ?? Kt</code>. Backend: 631.js:12401 <code>hd(e, { sport: i, area: t.area })</code> picks the top smart-schedule slot regardless of the caller's venue/time; 12466-12483 compares the caller's (stale) inputs to compute <code>current_success_pct</code>; 12489-12513 builds recommendations against the same stale inputs.	Organizer publishes a match whose price/capacity differ from anything shown for the slot they actually picked; "Now X% → Optimized Y%" and the recommendation list are misleading.	Add the relevant inputs (venue, date, start/end, price, capacity, skill, waitlist) to the effect dependencies with debounce, pass the chosen <code>venue_id/starts_at</code> through to <code>hd</code> , and show the final price/capacity in the quick-lane summary before publish.
F-ORG1-7 Medium	organizer/create	UX dead end	Quick lane in create shows an endless "analyzing" spinner and hidden defaults when the optimizer has nothing to say When <code>fetchMatch0optimization</code> resolves <code>null</code> or rejects, the "AI summary" card shows a spinner forever (<code>!Mt</code> is true and <code>Pt</code> is false). The Publish button still works and falls back to the custom-lane state (2461.js:1093-1098), but in the quick lane those fields are hidden, so the organizer publishes with <code>price 0</code> (2461.js:44 <code>pt = 0</code>, i.e. a free match), <code>skill "all"</code>, waitlist from format, none of which is displayed. Same for the court-booking path, where the spinner is not shown but the defaults are equally invisible. Evidence: 2461.js:750-754 — <code>Pt !Mt ? <ActivityIndicator/> : <summary></code>; 2461.js:92-94 — <code>.catch(() => { t It(null) });</code>; 631.js:12402-12403 — <code>const r = await hd(...); if (!r) return null</code>; and 12271 <code>if (!n) return null</code> (no slot for the sport/area, e.g. no venue supports it or <code>Js</code> returns empty). 2461.js:72 — with a court booking (<code>X</code>) the optimizer is skipped entirely (<code>It(null)</code>).	Free matches published unintentionally; loading state that never resolves.	Render an explicit "no recommendation available — using defaults: price X, players Y" state when <code>Mt</code> is null and not loading; disable quick publish until either the optimizer result or the user's explicit values are shown.
F-ORG1-8 Medium	organizer/quick	Financial control	organizer/quick posts a paid match without ever showing the price, capacity or duration when no history exists For a new organizer (the common case) the screen says "no data" and the Post button creates a public, auto-accept, 2.5 KWD-per-player, 90-minute, 10-player match. None of those values are displayed or confirmable before posting; the "duration" is never shown even when defaults exist. Evidence: 2470.js:373-426 — smart-defaults card shows price/squad/skill/waitlist only when <code>F</code> (smart defaults) is non-null, otherwise the single line <code>P("smartNoData")</code>; 2470.js:444-448 — payload is only <code>{sport, venue_id, starts_at}</code>; 631.js:3509-3513 <code>i = await Ki(...)</code>, <code>Ki</code> returns <code>null</code> unless <code>>= 3</code> comparable games exist (631.js:3479); fallback <code>\$i = 10 players / 2.5 KWD (3501-3503)</code>, 90 min (3513), <code>visibility: "public"</code>, <code>approval_mode: "auto"</code> (3531-3532).	Organizers unknowingly charge players (or set the wrong price/size); disputes and refunds.	Always render the effective values (price, players, duration, visibility, approval) in the summary card — from <code>Ki</code> when available, else from <code>\$i</code> — and require the organizer to see them before Post. Expose <code>\$i</code> via <code>fetchSmartDefaults</code> so client and server agree.
F-ORG2-10 Medium	organizer/match/[id]	Validation	Editing a match does not prevent moving it into the past or reducing capacity below held seats; court booking is not updated (a) An organizer can set the time to yesterday: the match instantly becomes "completed" (<code>Ji</code>, 631.js:3956), unlocking score submission and attendance marking with no play having occurred, while players are only told "time changed". (b) Capacity can be set below <code>ki()</code> (confirmed + active reserved holds, 631.js:2456) leaving paid-reserved players overbooked. (c) A venue court booking tied to the match keeps the old slot. Evidence: 631.js:3538-3591 <code>mockUpdateMatch</code> — validates only <code>ends_at <= starts_at</code> (3552) and <code>a.max_players < confirmed</code> (3567-3568, counts "confirmed" === <code>t.status</code> only). No <code>starts_at < Date.now()</code> check (creation path <code>Wi</code> has one at 631.js:3329 <code>if (i < Date.now() - 6e4) throw "E_MATCH_CANNOT_START_IN_PAST"</code>). <code>court_booking_id</code> is never read or updated. UI <code>DatePicker</code> at 2466.js:513-519 has no minimum date.	Fabricated results/attendance; overbooked paid seats; venue slot mismatch.	Reuse the creation validators (no past start, sane duration, 2..40 players), compare capacity to <code>ki(e)</code> , and either block time edits when <code>court_booking_id</code> is set or re-book the court.

ID / Severity	Interface	Risk Area	Identified Gap (with evidence)	Potential Impact	Recommended Remediation
F-ORG2-11 Medium	organizer/match/[id]	Financial control	Match can be cancelled while it is in progress, with automatic full refunds; reason optional Between <code>starts_at</code> and <code>ends_at</code> the organizer can cancel (or kick everyone) and every paid player is refunded 100% to their wallet, bypassing the <code>SEAT_REFUND_CUTOFF_HOURS</code> policy that applies to player-initiated cancellations (<code>mockGetSeatRefundQuote</code> , 631.js:3122-3138). No reason is required for a destructive, notified action. Evidence: 631.js:3600-3601 — <code>if (new Date(n.ends_at).getTime() < Date.now()) throw "E_THIS_MATCH_HAS_ALREADY_STARTED"</code> (checks <code>end</code> , not start); 3602 <code>const r = sanitizeText(a "", 200) "No reason provided";</code> 3622 <code>for (const e of o) await Br(n, e, !0);</code> (forced refund regardless of <code>isRefundEligible</code>). Same start/end confusion in <code>mockKickPlayer</code> 631.js:3266 and <code>mockCloseRegistration</code> 3661. UI cancel card (2466.js:639-690) does not require the reason field.	Organizer-side refund abuse after a match has effectively been played; venue still owed; disputes with no recorded reason.	Compare against <code>starts_at</code> for cancel/kick (or require an explicit "match did not take place" attestation after start), make the reason mandatory (min length) and store it in the audit entry (<code>match.cancelled</code> at 3627 currently logs only the game id).
F-ORG2-12 Medium	organizer/match/[id]	Financial control	Court-booking cancellation failure during match cancel is swallowed The match is marked cancelled and players refunded even when the linked venue booking could not be released; nobody is told and nothing is logged. Evidence: 631.js:3623-3626 — <code>if (n.court_booking_id) try { await Mr(t, n.court_booking_id, _) } catch {}</code> . <code>Mr</code> (631.js:6994-7001) throws on <code>E_BOOKING_NOT_FOUND</code> / <code>E_YOU_ARE_NOT_AUTHORIZED_TO_CANCEL</code> .	Organizer keeps paying for a court that is no longer used; venue sees a booking for a cancelled match.	Log an audit entry (<code>court.cancel_failed</code>) and surface a warning in the UI response (<code>{ court_released: false }</code>), or fail the cancel when the court cancel fails.
F-ORG2-15 Medium	organizer/match/[id]	Availability	Need-Player-Now radius changes and Concierge auto-invite re-send notifications with no rate limit or state check An organizer can blast nearby players repeatedly by toggling 10/20/30/All or tapping "Auto-invite" (which is shown for any not-ended match with open slots, 2466.js:472-478), including for matches that are closed for registration. Evidence: 631.js:4982-5001 <code>mockUpdateNeedPlayer</code> — every call does <code>n.npn_notifications_sent += await En(n, r) (4991)</code> ; UI fires it on each radius chip tap while active (2466.js:1296 <code>onPress: () => (v ? h(e) : j(e))</code>). The activation cap (<code>npn_activation_count >= 3</code> , 631.js:4956) only applies to <code>mockActivateNeedPlayer</code> . 631.js:11607-11656 <code>mockConciergeAutoInvite</code> has <code>zi</code> but no <code>status/starts_at/bookings</code> check and no cap; each press sends up to <code>autoInviteCount</code> <code>need_player</code> notifications (11621-11636) and increments <code>npn_notifications_sent</code> .	Notification spam / user churn; inflated <code>npn_notifications_sent</code> stats.	De-duplicate recipients per game (skip users already notified for this game), add a cooldown/cap to both endpoints, and reject auto-invite when <code>registration_closed_at</code> is set or the match is not scheduled.
F-ORG2-2 Medium	organizer/series/[id]	UX dead end	"End series" is irreversible, has no confirmation and leaves already-scheduled occurrences live One accidental tap permanently ends the series (no resume path once status is <code>ended</code>), while the generated future matches remain <code>scheduled</code> and still appear under "Upcoming occurrences". The organizer can no longer act on the series and must cancel each occurrence individually. "End" vs "Cancel" semantics are not explained anywhere on the screen (<code>endSeries: "End"</code> 909.js:2787). Evidence: 2471.js:180-186 — <code>onPress: () => J(() => (0, b.endSeries)(e, z.id))</code> fires immediately (no confirm card, unlike <code>Cancel</code> at 2471.js:189-240). 631.js:5751-5755 <code>mockEndSeries</code> only does <code>\$n(e, t, "ended")</code> ; it does not touch <code>Kn(e)</code> occurrences. 2471.js:148 hides all controls once <code>"ended" === T.status</code> .	Irrecoverable state from a mis-tap; orphaned scheduled matches that players can still join.	Add an inline confirmation card (same pattern as cancel) that explains "End stops generating new sessions; existing sessions stay scheduled", and consider allowing <code>resume</code> from <code>ended</code> or renaming to "Stop generating".
F-ORG2-4 Medium	organizer/series/[id]	Bug/crash	Series screen shows an infinite spinner when the series is missing or a fetch throws A stale/deleted series id (e.g. deep link, or a series removed by seeding) resolves to <code>T === null</code> , and any rejection in <code>fetchOrganizerMatches</code> / <code>fetchSeriesAnalytics</code> leaves <code>D === true</code> ; in both cases the user sees a spinner forever with no error and no back button rendered. Evidence: 2471.js:17-25 — <code>K</code> awaits <code>Promise.all([...])</code> with no <code>try/catch</code> , then <code>\$(t)...H(!1)</code> ; 2471.js:38 <code>if (D !T) return <ActivityIndicator></code> . 631.js:5843-5847 <code>mockGetSeries</code> returns <code>null</code> for an unknown id.	Dead end with no recovery except OS back gesture.	Wrap <code>K</code> in <code>try/catch</code> , render an error state with the back button when <code>!T</code> after loading, and use <code>Promise.allSettled</code> so analytics failure does not block the main view.
F-ORG2-5 Medium	organizer/match/[id]	Bug/crash	Organizer match screen also hangs on load errors (no try/catch around the initial fetch) Any thrown error (audience mismatch, storage failure) rejects the promise, <code>X(!1)</code> never runs, and the screen stays on the spinner. An unknown match id does resolve (<code>k === null</code> → generic "error" text at 2466.js:112-118) but that text has no back button either. Evidence: 2466.js:37-46 — <code>qe = useCallback(async () => { ... const t = await fetchOrganizerMatchScreen(o?.id ?? "", e); M(t.game) ... X(!1); })</code> with no <code>try/catch</code> ; 2466.js:107-111 renders the spinner while <code>Y</code> is true. Backend <code>Ni</code> (631.js:2649-2653) calls <code>ra(t, a.audience)</code> which throws <code>AUDIENCE_MISMATCH</code> (631.js:748-750) for a cross-audience viewer.	Unrecoverable loading state; unhandled promise rejection in console.	<code>try/catch</code> in <code>qe</code> , set an error state, and render the header/back button in both the error and spinner branches.

ID / Severity	Interface	Risk Area	Identified Gap (with evidence)	Potential Impact	Recommended Remediation
F-ORG2-6 Medium	organizer/match/[id]	Privacy	Participant lists of any match are returned to non-organizers by the screen endpoint The ownership check only strips <code>candidates/intel</code>; the participant roster (including pending applicants and waitlisted users of private games) is still returned to any authenticated caller who knows the game id. Evidence: 631.js:16548-16553 — <code>if (!(!a && !e && a.organizer_id === e)) return { game: a, participants: i, candidates: [], intel: [] }; — i = await Qi(t)</code> (631.js:3974-3983) is the full <code>Qt.bookings</code> split into <code>confirmed/reserved/pending/waitlist</code>, including <code>user_id</code>, <code>display_name</code>, <code>attendance</code>, <code>status</code>. The UI only hides it client-side: 2466.js:99 <code>Qe = ... k.organizer_id !== o.id → T.replace('/game/...')</code>.	Exposure of who applied to / was rejected from a match, attendance marks and no-show history to non-organizers.	Return an empty participants object (or throw <code>E_YOU_ARE_NOT_AUTHORIZED_TO_MANAGE_2</code>) when the caller is not the organizer, matching <code>mockGetMatchActivity</code> (631.js:3237-3242).
F-ORG2-7 Medium	organizer/series/[id]	Privacy	Series analytics endpoint (incl. revenue) has no caller/ownership check, and "Revenue" is an estimate, not real takings Any user can pull retention, fill-rate and revenue figures for any series id. The figure labelled "Revenue" (909.js:2784) ignores refunds, unpaid free-seat bookings and price changes over time. Evidence: 671.js:484 <code>e.fetchSeriesAnalytics = (o) => t.store.mockGetSeriesAnalytics(o);</code> (series id only); 631.js:5848-5850 <code>mockGetSeriesAnalytics = async (e) => { await ei(); const t = Qt.games.filter(...)</code> — no user parameter, no <code>zi/\$n</code>. Revenue is computed as <code>revenueKwd: s * (Wn(e)?.price_kwd ?? 0)</code> (631.js:5888), where <code>s</code> counts <code>confirmed-or-attended</code> bookings, not <code>Qt.payments</code>. <code>mockGetSeries</code> (5843) likewise takes <code>t</code> only for the <code>user_joined_upcoming</code> field.	Competitor/organizer financial data leak; organizer decisions based on inflated revenue.	Pass the caller id and enforce <code>organizer_id === caller admin</code> ; compute revenue from <code>Qt.payments</code> with <code>status === "paid"</code> for those games and label it "Estimated" if the estimate is kept.
F-ORG2-9 Medium	organizer/match/[id]	Data integrity	Pending requests can still be approved/rejected after the match has ended (UI shows the buttons; backend has no state check) State machine mismatch: every other mutation rejects finished/cancelled matches (<code>E_THIS_MATCH_HAS_ALREADY_STARTED / E_THIS_MATCH_CAN_NO_LONGER_BE</code>), approval does not. <code>mockRejectParticipant</code> (3199) has the same omission (harmless but inconsistent). Evidence: 2466.js:692-716 — the pending section renders <code>Approve/Reject</code> with no <code>!et && !Ze</code> guard (compare kick at 2466.js:810 <code>Xe && !et && !Ze</code>). 631.js:3158-3197 <code>mockApproveParticipant</code> checks only <code>E_MATCH_NOT_FOUND</code>, <code>zi</code> and capacity; no <code>status/ends_at</code> check, then <code>Hr(i, n.user_id, n.id)</code> (7202) creates a pending seat payment with a 2-minute hold and sends <code>payment_request + request_approved</code> notifications for a match that is already over.	Players billed for a past match; attendance/reliability stats polluted; confusing notifications.	In both mocks throw when <code>status !== "scheduled"</code> or <code>ends_at < now</code> ; in the UI hide the buttons when <code>et Ze</code> and show pending rows as read-only.
F-XC-12 Medium	Platform (performance)	Availability	Single 11.6 MB HTML file with 5.8 MB of base64 fonts and a 5.6 MB JS bundle Everything is downloaded, parsed and hydrated before any interface appears, including all icon font families and every admin screen for non-admin users. No caching headers, code splitting or lazy routes are possible in a data: URI bundle. Evidence: <code>index.html:30</code> — <code>window.__PLAYORA_ASSETS</code> with 42 inlined assets (24 TTF icon fonts totalling 5.8 MB); <code>index.html:130-2773</code> — one 5.6 MB script; 631.js:1745-1800 — <code>ti()</code> hydrates ~60 collections from storage before the first screen renders	Multi-second first load on mobile networks; memory pressure on low-end devices; poor SEO/Lighthouse; large localStorage hydration on each launch.	Ship a normal Expo web export (hashed static assets, HTTP caching, only the icon sets actually used), lazy-load the admin route group, and paginate collections instead of hydrating everything.
F-XC-13 Medium	Sign-in (phone)	Authorization	In-memory OTP rate limiter and console logging of codes OTP throttling (3 per 15 min, 10 per day) is held in a JavaScript <code>Map</code> that resets on page reload, so it provides no protection. When SMS is not configured the code is written to the browser console. Evidence: 631.js:761-775 — <code>_a = new Map(), ua = new Map()</code> rate-limit state; 642.js:6-10 — log provider prints <code>[sms:log] to ...\${o.slice(-4)}: \${t}</code> to the console	OTP brute force and code disclosure (moost while the static code 123456 is active, but it will matter once demo mode is disabled).	Enforce OTP issuance and verification server-side with persistent counters; never log codes.
F-XC-14 Medium	admin/* (deep links)	Ambiguity	No client-side role guard on admin routes; non-admins see misleading empty states instead of "access denied" Any signed-in user (including guests with ids <code>guest:*</code>) can open <code>/admin/awards</code>, <code>/admin/feed</code>, etc. by URL. The backend rejects the call, the screen swallows the error and shows "No fraud signals" (or a blank dashboard) as if the check had run. Evidence: 1809.js:60-66 — root guard only redirects unauthenticated users to sign-in; 1824.js:11-18 — <code>try { A(await fetchAwardFraudSignals(t.id)) } catch { A([]) }</code> then renders <code>EmptyState({ title: I("noFraud") })</code> on any error including <code>E_ADMINISTRATOR_AUTHORIZATION_REQUIRED</code>	Misleading state for testers and support staff; hides real failures from admins because access errors and empty results render identically.	Add an <code>AdminGate</code> wrapper in the admin layout that checks <code>profile.role</code> and renders an explicit "Administrator access required" screen; never map a thrown error to an empty list.
F-XC-3 Medium	Platform (all interfaces)	Authorization	Backend endpoint is selectable from localStorage and defaults to localhost The RPC base URL and mode are read from user-writable <code>localStorage</code> keys. Anyone (including a malicious browser extension or a stored-XSS payload) can redirect every RPC call, including the Bearer session token, to an arbitrary host. Evidence: 673.js:16 — <code>const n = "http://localhost:8090"</code>; 673.js:41-42 — <code>c = s("playora.backend.url") ?? n; const t = s("playora.backend");</code>	Session token exfiltration and data tampering once a real backend exists.	Bake the API origin into the build (<code>EXPO_PUBLIC_API_URL</code> , which <code>rpc.ts</code> already documents) and remove the runtime override, or restrict it to <code>__DEV__</code> builds.

ID / Severity	Interface	Risk Area	Identified Gap (with evidence)	Potential Impact	Recommended Remediation
F-ADM1-10 Low	admin/organizers, admin/conduct	Ambiguity	Dashboard counters are computed over all data while the lists beneath them are partition-filtered An admin can see "Pending review: 3" and an empty incident queue. Evidence: 631.js:12243-12261 — <code>mockGetConductAdminStats</code> counts <code>Qt.sanctions.unfiltered(pending: t.filter((e) => "pending_approval" === e.status).length)</code> whereas 631.js:12234 filters the queue through <code>cn</code> ; 631.js:5076-5092 (NPN), 5280-5305 (skill), 6002-6014 (growth), 6395-6400 (compat) likewise aggregate whole tables. Screen shows "Pending review" tile at 1826.js:85-90 above the queue at 1826.js:114-217.	Confusion and unactionable counters; may mask the hidden-world problem in F-ADM1-9.	Compute stats over the same partitioned set, or label the tiles "platform-wide".
F-ADM1-15 Low	admin/players	Bug/crash	Player search fires an unguarded request per keystroke and results go stale after an audience correction Out-of-order responses can render results for an earlier query; after moving a player to the other world the search card still shows the old audience label and the player vanishes from the intel list (now filtered out) with no explanation. Evidence: 1833.js:82-90 — <code>onChangeText: async (t) => { ... U(await adminFindPlayers(e.id, t)) ... }</code> (no debounce, no request-id guard); 1833.js:379-382 — after <code>adminCorrectAudience</code> only <code>Y()</code> (intel) is re-fetched, <code>G</code> (search results) is left untouched; 631.js:4401-4406 — search results are partition-filtered via <code>cn</code> .	Admin may believe the correction failed or apply it twice (second attempt correctly throws <code>E_THAT_PLAYER_IS_ALREADY_IN_THIS</code> , 631.js:4424, but only after the modal reopens).	Debounce and tag requests; after a successful correction clear/refresh the search results and show a toast explaining the player is now in the other world.
F-ADM1-17 Low	admin/players	Audit trail	Audience correction is logged with <code>logAudit</code> instead of <code>logAdminAudit</code> This admin action is missing the standard admin envelope, and the <code>target</code> is stored as the full user id rather than the truncated <code>actorRef</code> used elsewhere (inconsistent PII handling in the log). Evidence: 631.js:4431 — <code>await logAudit("admin.audience_corrected", actorRef(e), { target: t, ... })</code> versus every other privileged action in scope using <code>logAdminAudit</code> (e.g. 631.js:4587, 5444, 6724, 12187); 643.js:9-14 shows <code>logAdminAudit</code> adds <code>applicant</code> , <code>device</code> , <code>ip</code> fields.	Harder to filter admin actions; inconsistent identifier redaction.	Switch to <code>logAdminAudit("admin.audience_corrected" e, t, {...})</code> .
F-ADM1-18 Low	admin/players	Ambiguity	Player-intel empty state says "No matches yet" The key belongs to the organizer match list; on this screen it reads as "no search matches" while the list is actually the full skill-profile roster (possibly empty because of the partition, F-ADM1-9). Evidence: 1833.js:160-165 — <code>EmptyState({ icon: "people-outline", title: L("noOrganizerMatches"), body: "" })</code> ; 909.js:1046 <code>noOrganizerMatches: "No matches yet"</code> .	Admin misreads an empty roster as a search problem.	Add a dedicated key ("No player profiles in your world yet").
F-ADM1-22 Low	admin/venues	Validation	<code>mockAdminReviewVenue</code> treats any non approve/reject action as "suspend" and validates no transition Transitions such as rejected → approved and suspended → approved happen with no additional review; a typo action suspends a venue. Evidence: 631.js:6710 — <code>n.status = "approve" === a ? "approved" : "reject" === a ? "rejected" : "suspended"</code> ; no read of the current <code>n.status</code> ; 631.js:6695-6703 <code>mockGetPendingVenues</code> returns every venue profile regardless of status (misnamed) so the screen offers Approve on rejected and suspended venues (1834.js:162).	Accidental suspension via API; re-activation of rejected venues without re-review.	Allow-list the action, enforce a transition table, and rename/filter the query (or label the section "All venue registrations").
F-ADM1-25 Low	admin/conduct	Validation	<code>mockReviewSanction</code> has no state-transition validation; unknown actions overturn A rejected or already-overturned sanction can be re-activated via "approve", and any unrecognized action string overturns. Evidence: 631.js:12173 — <code>n.status = "approve" === a ? "active" : "reject" === a ? "rejected" : "overturned"</code> with no check of the prior status; 631.js:12168-12170 only checks existence. UI gates by status at 1826.js:176 and 194.	Sanction history becomes unreliable; a stale screen can flip a decision.	Allow-list actions and enforce: <code>pending_approval</code> → {active,rejected}, <code>active</code> → {overturned}.
F-ADM1-26 Low	admin/conduct	Authorization	"Restore account" skips the partition check, returns silently when nothing was lifted, and gives no success feedback Restore can act on a user in the other world (inconsistent policy), and if the count is 0 (e.g. sanction already overturned by another admin) the admin sees no message. The generic no-feedback pattern applies to all conduct and venue actions (1834.js:33-42). Evidence: 631.js:12191-12199 — <code>mockRestoreUser: (await ei(), await sn(e)); let a = 0; for (const i of Qt.sanctions) ... — no _n(e, un(t), ...)</code> unlike <code>mockReviewSanction</code> at 631.js:12168; returns <code>a</code> (count); 1826.js:209 — <code>G(() => restoreUser(e.id, t.user_id))</code> ignores the return value; 1826.js:30-39 — <code>G</code> only refreshes, never alerts success.	Inconsistent partition enforcement; admins repeat actions unsure whether they took effect.	Add the <code>_n</code> check, and show a toast with the lifted count (or "nothing to restore").
F-ADM1-27 Low	admin/conduct	UX dead end	Evidence URL is rendered as inert text The link is styled as a link (accent colour) but cannot be opened, so the reviewer cannot examine evidence before approving a ban. Evidence: 1826.js:163-167 — <code>!!t.evidence_url && Text({ style: [caption, { color: o.accentText }], children: t.evidence_url })</code> (no <code>onPress/Linking</code>).	Decisions made without viewing evidence; frustration.	Wrap in a Pressable calling <code>Linking.openURL</code> after validating the scheme.

ID / Severity	Interface	Risk Area	Identified Gap (with evidence)	Potential Impact	Recommended Remediation
F-ADM1-28 Low	admin/conduct	Bug/crash	Violation category "dangerous_play" maps to a non-existent i18n key The screen re-derives the key instead of using the <code>labelKey</code> the rules module already provides, and gets it wrong for this category. The raw key (or blank, depending on <code>useT</code> fallback — needs confirmation) is shown to the admin. Evidence: 1826.js:147 — <code>y(\vio\$(l(t.category))\)</code> with 1826.js:249-253 <code>I = (e) => e.split("_").map(capitalize).join("") - "vioDangerousPlay"</code> ; 656.js (raw bundle) <code>{category: 'dangerous_play', labelKey: 'vioDangerous'}</code> ; 909.js has <code>vioDangerous</code> : but no <code>vioDangerousPlay</code> : (grep count 0).	Untranslated/unknown label on yellow-card incidents for dangerous play in both locales.	Look up <code>VIOLATION_CATEGORIES.find(c => c.category === t.category)?.labelKey</code> (as <code>sanctionMeta</code> does for types) instead of string-building.
F-ADM1-30 Low	admin/venues, admin/conduct	Ambiguity	Shared loading flag makes every action button on every card spin at once While one venue/sanction is being processed all other cards' buttons show a spinner, so the admin cannot tell which record is being acted on and cannot act on another. Evidence: 1834.js:166, 174, 182, 252 — <code>loading: G</code> (single state, 1834.js:14); 1826.js:182, 188, 198, 208 — <code>loading: 0</code> (1826.js:14).	Minor confusion; double-tap risk is mitigated but progress attribution is lost.	Track <code>busyId</code> (venue/sanction id) and pass <code>loading: busyId === t.id</code> .
F-ADM1-31 Low	admin/organizers, admin/application/[id]	Privacy	Application list and detail expose hashed identity references and full contact PII to the client The list call ships mobile, email and document hashes for every applicant although the list UI needs none of it; hashes of national ID numbers are a re-identification aid. Evidence: 631.js:4550-4558 — <code>pn</code> returns <code>Object.assign({}, e, {...})</code> (whole application record including <code>phone_hash</code> , <code>id_doc_ref</code> , <code>business.registration_number_ref</code> , <code>license_ref</code> , <code>mobile</code> , <code>email</code>); 631.js:4562-4565 — the list endpoint returns the same shape for every application; 1832.js shows only <code>display_name/full_legal_name</code> .	Larger PII surface in memory/devtools for a list view.	Return a slim DTO from <code>mockGetOrganizerApplications</code> and strip <code>*_ref/*_hash</code> fields from the detail DTO.
F-ADM1-5 Low	admin/application/[id]	Ambiguity	Reason placeholder promises the note is "sent to the applicant", but on Approve it is not For approve the text goes into internal <code>admin_notes</code> and the applicant notification carries an empty note; for reject/suspend the same field is sent externally. One field, two behaviours. Evidence: 909.js:1189 — <code>decisionReasonPlaceholder</code> : "Add a reason or note (sent to the applicant)"; 631.js:4588 — <code>await wn(n.user_id, n.id, "approved", "")</code> (empty note); 631.js:4583 stores the typed text only in <code>admin_notes</code> .	Admins may write internal remarks that get sent to rejected applicants, or write welcome notes that approved applicants never see.	Split into "internal note" and "message to applicant" fields, or change the placeholder per action and pass the note to <code>wn</code> on approve.
F-ADM1-8 Low	admin/application/[id]	Validation	<code>mockReviewApplication</code> performs no state-transition validation; unknown actions fall through to "suspend" Any caller (or a stale screen) can approve an already-approved record, "request info" on a rejected one, or pass a typo action string and suspend an organizer. Evidence: 631.js:4579-4618 — <code>if ("approve" === a) ... else if ("reject" === a) ... else if ("request_info" === a) ... else { ... n.status = "suspended" ... }</code> ; no read of <code>n.status</code> before writing; only the UI gates actions (1823.js:352-361 <code>0(status)</code>).	Inconsistent status histories and accidental suspensions via API misuse.	Validate <code>a</code> against an allow-list and enforce the same transition table as <code>D()</code> server-side (throw <code>E_INVALID_TRANSITION</code>).
F-ADM2-10 Low	admin/concierge	Validation	Concierge rule bounds differ between UI and server The server accepts values (up to 168 h) the UI cannot display or set back, and non-step values for <code>skillTolerance</code> ; a value set by another path becomes unreachable from the stepper. Evidence: 1825.js:175 — <code>notifyLeadHours</code> <code>... max: 72</code> ; 631.js:11703 — <code>Math.max(1, Math.min(168, Math.round(a.notifyLeadHours)))</code> ; 631.js:11702 <code>skillTolerance</code> is clamped but not rounded to the 0.5 UI step.	Minor inconsistency; an out-of-range persisted value cannot be brought back within UI range in one tap.	Share one bounds table between screen and store; round to step server-side.
F-ADM2-11 Low	admin/concierge	Ambiguity	Concierge stats show 0% accuracy when nothing has been evaluated With zero evaluated games the tile reads "0%" accuracy, indistinguishable from a model that is 0% accurate; the <code>evaluated</code> count that would disambiguate is returned but not rendered (admin/demand and admin/optimizer do render it). Evidence: 631.js:11736 — <code>accuracyPct: 0 ? Math.round(100 - i / o) : 0</code> ; the screen (1825.js:83-99) shows <code>plansGenerated</code> , <code>autoInvites</code> , <code>accuracyPct</code> , <code>avgPredictedFill</code> , <code>avgActualFill</code> but never <code>predictions/evaluated</code> .	Misleading KPI.	Render <code>evaluated</code> and show "—" when it is 0.
F-ADM2-12 Low	admin/demand	Ambiguity	Demand "model performance" tiles do not reflect the weights being tuned The screen presents accuracy/avg-predicted tiles directly above the weight steppers, implying a feedback loop, but changing any weight cannot move those tiles. Evidence: 631.js:12366-12384 — <code>mockGetDemandModelStats</code> computes <code>fillAccuracyPct</code> from <code>td(e)</code> ; 631.js:11591-11605 — <code>td</code> uses <code>timeScore/dayScore/weatherFor/availability</code> and constants <code>demand: 0.6</code> , <code>venue: 0.7</code> , never <code>fd()</code> ; the weights are consumed only at 631.js:12268 (<code>mockGetDemandPrediction</code>).	Admin tunes weights against a metric they do not influence.	Either compute the accuracy from the weighted predictor or label the tiles as "baseline fill model".

ID / Severity	Interface	Risk Area	Identified Gap (with evidence)	Potential Impact	Recommended Remediation
F-ADM2-16 Low	admin/insights	UX dead end	Export delivery on web relies on blob URL revoked synchronously; no path on native The anchor is never attached to the DOM and the blob URL is revoked in the same tick as <code>click()</code> ; Chrome tolerates this, Firefox/Safari can abort the download (needs confirmation in a real browser — CSP fetch directives do not govern <code><a download></code> navigations, so the CSP itself is not the blocker). On a non-web platform no file is produced at all, yet the "Export CSV / <filename>" success alert is still shown. Evidence: 1830.js:81-87 — <code>if ("undefined" != typeof document) { const e = new Blob([o], {type:"text/csv"}), t = URL.createObjectURL(e), r = document.createElement("a"); r.href = t; r.download = l; r.click(); URL.revokeObjectURL(t); } r.default.alert(w("biExport"));</code> <code>\$({}); index.html CSP: default-src 'none' ... connect-src 'none'.</code>	Users may see a success alert without a file.	Append the anchor, revoke via <code>setTimeout</code> , and gate the success alert on the download path actually running; on native use a share sheet.
F-ADM2-18 Low	admin/insights	Ambiguity	No loading indicator while re-fetching with new filters; stale data shown After tapping "7 days" or a sport chip the old 90-day/any-sport numbers remain on screen until the promise resolves, with no spinner or dimming. Evidence: 1830.js:36 — <code>if (q && !V)</code> spinner only when no data yet; 1830.js:107/123 change <code>L/_</code> and trigger <code>ee</code> (deps 1830.js:30) but the previous <code>V</code> stays rendered; only 1830.js:172 <code>biGenerated</code> timestamp hints at freshness.	Screenshots/decisions made on the wrong filter window.	Show an inline <code>ActivityIndicator</code> or dim the content while <code>q</code> is true.
F-ADM2-19 Low	admin/insights	Ambiguity	Health tab and CAC/time-to-fill tiles are simulated values presented alongside real KPIs Hard-coded constants and a "soon" placeholder live inside a screen labelled Business Intelligence; the only differentiator is a 9pt 70%-opacity caption, and the CSV export does not carry the flag. Evidence: 631.js:10961-10964 — <code>errorRatePct: 0.4, uptimePct: 99.95, apiP95Ms: 180, simulated: !0;</code> 1830.js:694-711 shows them with a 9pt <code>biSimulated</code> sub-label; 1830.js:336/502 <code>biModeled</code> ; 1830.js:456 renders <code>premiumSoon</code> ("Premium conversion (soon)") as a plain caption.	Simulated uptime/error-rate can be quoted as fact.	Move simulated metrics behind a clearly labelled "Demo" section or hide when <code>simulated</code> is true; remove the "soon" caption.
F-ADM2-21 Low	admin/feed	Ambiguity	Feed screen maps every error to "Access denied" A persistence failure or thrown bug is reported as an authorization problem; the body text is just the screen title. Evidence: 1828.js:16-18 — <code>catch { S(!0) };</code> 1828.js:48-53 — <code>EmptyState { title: M("accessDenied"), body: M("feedAnalyticsTitle") }.</code>	Misdirected troubleshooting.	Inspect the error code (<code>storeErrorText</code>) and show a generic error with retry for non-auth failures.
F-ADM2-22 Low	(tabs)/profile admin panel, admin/feed, admin/funnel, admin/awards	Authorization	Role gating is inconsistent between the panel, the screens and the backend An analyst who types <code>/admin/feed</code> gets a working screen with no link to it; typing <code>/admin/funnel</code> or <code>/admin/awards</code> yields the misleading "no data" states (F-ADM2-6); <code>/admin/concierge</code> etc. yields "Verification required". Backend authorization is correct, but the client presents three different outcomes for the same "not allowed" situation. Note: the panel section label is the hard-coded string "ADMIN" (1805.js:755), not <code>i18n</code> , so it stays English in Arabic. Evidence: 1805.js:97 <code>Je = "admin" === M?.role;</code> 1805.js:752 <code>(Je "analyst" === M?.role) &&</code> shows only Insights to analysts; 1805.js:762 nests all other links under <code>Je &&</code> ; backend: feed uses <code>Ms</code> (631.js:15209, analyst allowed) while funnel/awards/concierge/demand/optimizer use <code>Sn</code> (admin only). None of the seven screens checks <code>profile.role</code> client-side; 1830.js:57 only uses it for a "read-only" badge.	Confusion; support tickets; the analyst role's actual entitlements are undiscoverable.	Decide per screen whether analyst is allowed, gate consistently on both ends, and render a single "admin only" state with the same copy; localise the section label.
F-ADM2-23 Low	(tabs)/profile admin panel, admin/organizers	UX dead end	admin/players is reachable only via an unlabelled icon in the Organizer approvals header "Player intelligence" (909.js:1294) is not listed in the ADMIN panel and is discoverable only by tapping an unlabeled icon on a different admin screen. Evidence: 1805.js:756-818 — no <code>J.push("/admin/players")</code> ; the only link is 1832.js:62 <code>onPlayers: () => D.push("/admin/players")</code> ; rendered as a <code>people-outline</code> icon button in the organizers header; route exists (routes.txt:20, registered 1809.js:136).	Feature is effectively hidden.	Add an <code>0</code> row for <code>adminPlayersTitle</code> → <code>/admin/players</code> under the <code>Je &&</code> block in 1805.js.
F-ADM2-24 Low	admin/demand, admin/optimizer	Ambiguity	Accuracy/rate stats denominators mix statuses and can hide zero-data cases Zero-data and zero-performance are indistinguishable in four tiles, and the demand screen's "Decisions" tile is actually the count of demand predictions logged. Evidence: 631.js:12376 and 12589 — actual fill counts bookings with <code>"confirmed" === status "attended" === attendance</code> , so an attended-but-cancelled row is counted; <code>acceptRatePct: Qt.demandLog.length ? ... : 0</code> and <code>accept_rate_pct: p ? ... : 0</code> show 0% when there are no decisions; 1827.js:94-95 labels the prediction count with <code>perfPredictions</code> ("Decisions", 909.js:2259).	Misread KPIs.	Show "—" for zero denominators; give the demand tile its own label; make the attended/confirmed predicate mutually exclusive.

ID / Severity	Interface	Risk Area	Identified Gap (with evidence)	Potential Impact	Recommended Remediation
F-ADM2-7 Low	admin/concierge, admin/demand, admin/optimizer, admin/insights	Ambiguity	Error state on config screens shows the organizer-verification message Any failure (network, persistence, role) shows the heading "Verification required" with body "Admin", a message written for unapproved organizers. It does not say what failed or offer retry. Evidence: 1825.js:45-51, 1827.js:45-49, 1831.js:45-49 — <code>EmptyState { title: h("organizerGateTitle"), body: h("adminPanel") }</code> ; 909.js:1201 <code>organizerGateTitle: "Verification required"</code> , 909.js:1158 <code>adminPanel: "Admin"</code> ; 1830.js:49-53 uses <code>organizerGateTitle + biTitle</code> .	Admins misread a transient failure as a permissions problem; analysts navigating by URL get a message about "verification" instead of "admin only".	Add dedicated <code>adminOnlyTitle/adminLoadFailed</code> strings, distinguish <code>E_ADMINISTRATOR_AUTHORIZATION_REQUIRED</code> from other errors via <code>storeErrorText</code> , and add a retry button.
F-ADM2-8 Low	admin/awards	Ambiguity	Awards empty-state body uses the leaderboard subtitle The body text under "No suspicious voting detected." reads "Top award winners", which is copy for a different screen. Evidence: 1824.js:60-63 — <code>EmptyState { title: I("noFraud"), body: I("leaderboardSub") }</code> ; 909.js:1374 <code>leaderboardSub: "Top award winners"</code> .	Confusing copy on a control report.	Use a dedicated key (e.g. <code>noFraudBody</code>) or omit the body.
F-ADM2-9 Low	admin/awards	UX dead end	Fraud signals have no review workflow (no dismiss/acknowledge/revoke from the list) Every flagged pair reappears on every visit forever; there is no way to mark a signal reviewed, and <code>mockRevokeAward</code> (631.js:14914) is not reachable from this screen. Whether <code>/awards/[gameId]</code> (route exists, <code>routes.txt:22</code>) exposes a revoke action for admins needs confirmation. Evidence: 1824.js:98-109 — the only action per row is <code>B.push(/awards/\${t.match_id})</code> ; 631.js:14927-14957 recomputes signals from <code>Qt.awardVotes</code> on every visit with no reviewed/dismissed state.	The list grows monotonically and the admin cannot tell new signals from ones already investigated.	Persist a per-(match,voter,nominee) review status with reason, filter reviewed rows by default, and expose revoke from the row.
F-ORG1-10 Low	organizer/index	Bug/crash	Dashboard match card dereferences venue without a guard A game whose <code>venue_id</code> is no longer resolvable (e.g. created against a demo venue and later loaded with demo mode off, or any future venue removal) makes the whole dashboard throw on render. Evidence: 2465.js:534 — <code>children: [t.venue.name, " · ", formatGameTime(t.starts_at)]</code> ; 631.js:2615-2619 <code>Ii = (e, t, a) => { const i = t.get(e.venue_id); ... venue: i }</code> (undefined when the venue is not in the map); 631.js:2447-2451 — <code>fi()</code> includes demo venues <code>Ta</code> only while <code>Fa()</code> is true, while <code>gi()</code> returns persisted <code>Qt.games</code> regardless.	Organizer dashboard becomes unusable for that organizer.	Use <code>t.venue?.name ?? "-"</code> and have <code>Ii</code> fall back to a placeholder venue.
F-ORG1-15 Low	organizer/apply (backend <code>mockGetMyOrganizerApplication</code>)	Privacy	Applicant receives the raw application record (risk flags, hashes, reviewer identity) Fraud heuristics and internal reviewer data are exposed to the applicant (the UI only renders some fields, but the data is on the client). Evidence: 631.js:4455 — <code>nn(e) ?? null</code> returns the stored object verbatim, which includes <code>risk_flags</code> (4519, e.g. <code>duplicate_phone</code> , <code>duplicate_document</code>), <code>phone_hash</code> (4518), <code>id_doc_ref</code> (4514), <code>reviewed_by</code> (4520) and <code>admin_notes</code> (4522).	Applicants can learn how duplicate detection works and who reviewed them.	Return an applicant DTO limited to status, timestamps, rejection reason, reapply date and any applicant-facing message.
F-ORG1-18 Low	organizer/create, organizer/quick, organizer/new (backend <code>Wi</code>)	Validation	Match input validation gaps in <code>mockCreateMatch</code> (clamping instead of rejecting, unvalidated enums, NaN capacity) Server accepts semantically invalid values or silently rewrites money fields; client feedback is uninformative. Evidence: 631.js:3331 — <code>if (t.max_players < 2 t.max_players > 40) throw</code> (both comparisons are false for <code>NaN/undefined</code> , so they pass); 3332 <code>r = Math.max(0, Math.min(100, Number(t.price_kwd ?? 0)))</code> (negative or non-numeric silently becomes 0/ <code>NaN</code> → 0, >100 becomes 100); 3334 waitlist silently clamped; 3403-3405, 3413, 3415 — <code>sport</code> , <code>format</code> , <code>skill_level</code> , <code>visibility</code> , <code>approval_mode</code> stored verbatim with no allow-list; 3379 checks the venue exists but not that it supports <code>t.sport</code> ; no maximum duration (3408). <code>mockQuickCreateMatch</code> 3507 also only checks existence. Client: 2461.js:220 turns any non-numeric price into 0 with no message; 2461.js:1026-1034 alert bodies are bare field labels (<code>alert(G("error"), G("fieldTitle"))</code>), and there is no client check for a start time in the past.	Corrupt records (e.g. <code>max_players: NaN</code> , <code>visibility: "x"</code>) and mispriced matches with no indication to the organizer.	Validate enums against allow-lists, require <code>Number.isInteger(max_players)</code> , reject (not clamp) out-of-range price/waitlist, check <code>venue.sports.includes(sport)</code> , and give the client validation messages that state the rule.
F-ORG1-20 Low	organizer/create, organizer/quick, organizer/new, organizer/smart-schedule	Ambiguity	Errors swallowed into "empty" states that are indistinguishable from real emptiness Network/storage failures are presented as "nothing here", and a bad booking id is only discovered at publish time via the backend error. Evidence: 2461.js:66-70 <code>fetchVenues().then(_t).catch(() => {})</code> and 2461.js:668-672 renders the search placeholder string (<code>searchVenuePlaceholder</code>) as the empty-list body; 2470.js:22-26 and 2468.js:37-41 <code>.catch(() => W([]))</code> → "quickNoVenues"; 2472.js:16-21 <code>catch { K([]) }</code> → <code>EmptyState "noRecommendations"</code> ; 2461.js:153-155 — an error fetching the application redirects to <code>/organizer/apply</code> exactly as if the user were not approved; 2461.js:195-211 <code>fetchBooking</code> has no catch (unhandled rejection) and a <code>null</code> booking is ignored while <code>court_booking_id</code> is still sent (1107).	Users conclude there are no venues/slots and abandon the flow; support cannot distinguish outage from data.	Keep an <code>error</code> state per loader and render a retry affordance; validate the booking id up front and show a message when it cannot be loaded.

ID / Severity	Interface	Risk Area	Identified Gap (with evidence)	Potential Impact	Recommended Remediation
F-ORG1-21 Low	organizer/apply	Ambiguity	Rejected-application status card and "Reapply now" button are unreachable Whenever a rejected applicant is eligible to reapply, <code>ze</code> is true and the form is shown directly, so the "Rejected" headline and the <code>reapplyNow</code> button never render; the only rejected-state UI the user sees is the inline reason card (2459.js:83-98). While the cooldown is active they see the status card with "reapply in N days" but the form fields are hidden, so they cannot prepare edits. Evidence: 2459.js:43-44 — <code>Re = "rejected" === F?.status && (!F.reapply_after reapply_after <= now), ze = !F Re</code> ; 2459.js:69-77 — status card <code>B</code> renders only when <code>!ze && F</code> ; 2459.js:495-501 — inside <code>B</code> , <code>"rejected" === e.status && (d > 0 ? "reapplyIn" : <Button reapplyNow onPress= {onReapply}/>)</code> .	Minor confusion; dead code path.	Show the status card with the reason and an explicit "Reapply" button that reveals the form, rather than switching on eligibility implicitly.
F-ORG1-22 Low	organizer/index	Data integrity	Demo seeding injects fabricated matches/ratings into a real organizer's dashboard analytics Reputation score, tiers and completion rates shown to (and about) the first real organizer are computed on seeded fixtures; the auto-approval part is already documented, but the metrics contamination is not. Evidence: 631.js:4092-4112 — <code>tn()</code> runs when <code>Fa()</code> (demo world) and seeds via <code>an()</code> for the current user (<code>Qt.orgSeedUid = t, Qt.games.some((e) => e.organizer_id === t) ...</code>); 631.js:4286-4287 persists seeded games/bookings/ratings; 631.js:2451 <code>gi()</code> merges demo games <code>La</code> into every organizer query; 2465.js:28-33 stats/reputation/ratings are computed over <code>gi()</code> (631.js:3986, 4021, 4055). <code>Fa</code> (631.js:1682) is true unless <code>PLAYORA_DEMO === "0"</code> , i.e. on by default.	Misleading trust signals shown to players if the demo flag remains on in production.	Tag seeded rows (<code>seeded: true</code>) and exclude them from stats/reputation, or ensure <code>PLAYORA_DEMO=0</code> is enforced in the production build.
F-ORG1-4 Low	organizer/new	UX dead end	"Who picks spots" choice in organizer/new is silently discarded The field is sent but never read or stored; the option has no effect. Evidence: 2468.js:572-586 — selector <code>players / organizer (newPlayersChoose/newOrganiserAssigns)</code> ; 2468.js:755 — <code>{ formation_key: xe ?? void 0, spot_assignment: je }</code> ; <code>grep spot_assignment 631.js</code> → no occurrence; <code>Wi</code> only reads <code>t.formation_key</code> (631.js:3446-3455).	Organizer believes they control lineup assignment; behaviour is always the default.	Persist <code>spot_assignment</code> on the game and honour it in lineup logic, or remove the control.
F-ORG1-9 Low	organizer/index, organizer/apply	Availability	Dashboard and Apply screens spin forever if any loader rejects Any rejection (including from a side effect inside a "get") leaves the loading flag set and the user on an infinite spinner with no error, no retry. Evidence: 2465.js:25-36 — <code>await Promise.all([...6 fetches...]); ... \$(!1)</code> with no try/catch; the spinner is shown while <code>N</code> is true (2465.js:115-119). 2465.js:19-24 — <code>fetchMyOrganizerApplication(...).then(...)</code> with no <code>.catch</code> , and the whole screen renders a spinner until <code>Q</code> becomes <code>true</code> (2465.js:44-48). 2459.js:29-35 — <code>we</code> awaits <code>fetchMyOrganizerApplication</code> with no catch; spinner while <code>U</code> (2459.js:37-41). <code>mockGetOrganizerMatches</code> performs writes during the read (631.js:3966-3968 <code>Oi, ur</code>) which can throw.	Organizer locked out of the dashboard/apply screen with no feedback.	Wrap the loaders in try/catch, clear the loading flag in <code>finally</code> , and render an error state with retry (compare 2472.js:16-21 which at least clears loading).
F-ORG2-13 Low	organizer/match/[id]	Financial control	Series-cancel/kick/cancel refunds go to the in-app wallet, never to the original payment method (needs confirmation) KNET/card payers refunded after an organizer kick or cancel receive wallet credit only; there is no gateway refund path or admin visibility of the liability created. Evidence: 631.js:7244-7285 <code>Br</code> → <code>await ac(t, i.amount_kwd, i.id)</code> ; 631.js:15830-15848 <code>ac</code> posts <code>kind: "refund"</code> from <code>A.acct.platform("refunds")</code> to <code>A.acct.available(e)</code> (wallet ledger) and logs <code>wallet.refund_credited</code> .	Regulatory/consumer-protection exposure if wallet credit cannot be withdrawn; growing platform refund liability.	Confirm the intended policy; if wallet-only is intentional, state it in the cancel/kick confirmation copy and in the <code>refund_issued</code> notification.
F-ORG2-14 Low	organizer/match/[id]	Data integrity	Attendance can be marked at any time (before kick-off, on cancelled matches) and is never written to the match activity log Attendance (which feeds reliability/no-show stats used in <code>intel</code> badges, 2466.js:721-745) can be set or flipped at any time by the organizer with no in-app trace and no lock once results are final. Evidence: 631.js:3638-3652 <code>mockSetAttendance</code> — after <code>zi(n, t)</code> it sets <code>r.attendance = i</code> with no <code>status/ends_at</code> check, no <code>Xt</code> lock, and only <code>logAudit("attendance.marked", ...)</code> (3648) — no <code>Yi(...)</code> match-event, so the on-screen "Activity log" (2466.js:1151-1170, six action types) never shows attendance changes. The UI only exposes the toggles when <code>et</code> (2466.js:765-766), and they remain toggleable indefinitely after the score is final.	Reliability scores and series "avg attendance" (631.js:5880) can be manipulated retroactively without visibility to players or admins.	Reject attendance before <code>ends_at</code> and on cancelled games, record an <code>attendance_marked</code> match event with target/value, and freeze attendance N hours after score submission (or require admin override).
F-ORG2-16 Low	organizer/match/[id]	Audit trail	Concierge replacement read writes an audit entry on every render, crowding the audit ring buffer A pure read produces an audit line each time the screen opens, while genuine privileged events (attendance, kicks) share the same bounded log. Evidence: 631.js:11672-11675 — <code>mockGetConciergeReplacements</code> ends with <code>await logAudit("concierge.replacement_suggested", ...)</code> ; it is fetched on every mount / open-slot change by component <code>G</code> (2466.js:1546-1554).	Real audit entries are evicted sooner; log noise hides real activity.	Remove the audit call from the read path (keep it on <code>mockConciergeAutoInvite</code>).

ID / Severity	Interface	Risk Area	Identified Gap (with evidence)	Potential Impact	Recommended Remediation
F-ORG2-17 Low	organizer/match/[id]	Bug/crash	Errors from the four secondary panels are swallowed and rendered as "empty" states A failing backend call is indistinguishable from "nothing to show"; there is no retry and no console/analytics report. Evidence: 2466.js:57-59 activity log <code>.catch(() => { t && Me([]); })</code> → renders <code>noActivityYet</code> ; 2466.js:1549 <code>fetchConciergeReplacements(...).catch(() => [])</code> → renders "—"; 2466.js:1624-1625 <code>fetchReplacementState(...).catch(() => null)</code> → component <code>U</code> returns <code>null</code> (1640), so the whole "Smart replacement" card disappears; 2466.js:1830 <code>fetchGameGroups(...).catch(() => c([]))</code> → card hidden. Backend <code>mockGetGameGroups</code> throws the untranslated strings <code>"not_found"</code> / <code>"forbidden"</code> (631.js:13494-13495; same pattern at 8672, 12653).	Organizer believes there is no activity / no replacement system for the match; support cannot diagnose.	Keep a per-panel error state with a "Retry" affordance, and map the group-booking error strings to <code>E_*</code> codes handled by <code>storeErrorText</code> .
F-ORG2-18 Low	organizer/series/[id]	Authorization	Series screen renders management controls for non-organizers and relies on backend errors Any user reaching <code>/organizer/series/<id></code> (deep link) sees a working-looking management UI; pressing a button yields a generic authorization alert. <code>mockGetSeries</code> (631.js:5843) returns the template to anyone. Evidence: 2471.js:148-188 — <code>Pause/Resume/End/Cancel</code> are gated only by <code>"ended" !== T.status</code> ; there is no <code>T.organizer_id === z.id</code> check and no redirect (contrast 2466.js:99-104). Backend <code>\$n</code> (631.js:5729) does reject with <code>E_YOU_ARE_NOT_AUTHORIZED_TO_MANAGE_3</code> .	Confusing UX; enumeration of series ids exposes organizer settings (price, notes, weekdays).	Mirror the match screen: redirect to a public series/game view when <code>T.organizer_id !== z.id</code> , and have <code>mockGetSeries</code> strip organizer-only fields for non-owners.
F-ORG2-19 Low	organizer/series/[id]	Ambiguity	Cancel-series dialog reuses match wording; dialog closes even when the cancel failed The "keep" option is labelled for a single match, and after a failed cancel (e.g. <code>E_ORGANIZER_APPROVAL_REQUIRED...</code>) the reason text and dialog are discarded, forcing re-entry. Evidence: 2471.js:217-223 — secondary button <code>title: P("keepMatch")</code> ("Keep match", 909.js:1069) inside <code>cancelSeriesTitle</code> card; 2471.js:229 <code>z && e && await J(() => cancelSeries(e, z.id, E))</code> , <code>F(!1)</code> ; — <code>J</code> catches and alerts, so <code>F(!1)</code> always runs.	Minor confusion; lost input on error.	Add a <code>keepSeries</code> string and only close the dialog on success (return a boolean from <code>J</code>).
F-ORG2-20 Low	organizer/series/[id]	UX dead end	Upcoming-occurrence list is capped at 12 with no overflow affordance; no series-level edit UI although the backend supports it A daily series quickly exceeds 12 upcoming sessions; occurrences 13+ are unreachable from this screen. Bulk changes (time, price, capacity) require editing each occurrence individually. Evidence: 2471.js:252 <code>X.slice(0, 12).map(...)</code> while the header shows the full count (2471.js:242 <code>formatNumber(X.length)</code>); <code>editSingleHint</code> (909.js:2793 "Tap a session to edit or cancel just that occurrence") is the only edit path. <code>mockEditFutureOccurrences</code> (631.js:5687) exists in 631/671 but no screen module references it (grep over <code>pretty/*.js</code>).	Organizer cannot reach or manage later sessions; the "edit future occurrences" backend is dead code.	Add "Show all" pagination and an "Edit future sessions" form wired to <code>editFutureOccurrences</code> .
F-ORG2-21 Low	organizer/match/[id]	Ambiguity	Sanction UI: success toast says "issued" even when the sanction only went to admin review; flag shown on organizer's own row Organizer believes a ban is in force when it is pending; a self-flag button that can only error. Evidence: 2466.js:2061 <code>r.default.alert(u("sanctionIssuedToast"), "")</code> after <code>issueSanction</code> , although for <code>ban/suspension</code> the backend stores <code>status: "pending_approval"</code> (631.js:12122) and notifies admins instead of the player; 2466.js:801-810 the flag (<code>flag-outline</code>) button has no <code>e.user_id !== k.organizer_id</code> guard (the kick button at 815 has one), and the backend replies <code>E_YOU_CANNOT_SANCTION_YOURSELF</code> (631.js:12104).	False assurance to organizer; needless error path.	Branch the toast on <code>result.status</code> ("Sent for admin review"), and hide the flag button for the organizer's own row.
F-ORG2-22 Low	organizer/match/[id]	Data integrity	Score submission — validation and locking are sound; no correction path exists (needs confirmation) There is no admin or organizer route to correct a typo (grep shows no <code>mockCorrectScore/mockResetScore</code>), and the wrong result immediately fans out <code>match_score</code> notifications (631.js:3701-3712) and feeds leaderboards/compat. Evidence: 631.js:3682-3690 — rejects cancelled matches, matches not yet ended, already-final results (<code>E_THIS_RESULT_IS_FINAL</code>) and non-integer / out-of-range (0-99) values; UI input strips non-digits (2466.js:396) and the button is disabled until both fields are filled (2466.js:439). Once <code>score_submitted_at</code> is set the UI shows the number read-only (2466.js:362-374) and copy says "This cannot be undone" (909.js:255).	Permanently wrong results with no remediation path.	Add a confirmation step showing the entered score before submit, and an admin-only corrector endpoint that logs <code>match.score_corrected</code> via <code>logAdminAudit</code> .
F-ORG2-23 Low	organizer/match/[id]	Audit trail	Match cancel/kick audit entries do not carry the reason; reject carries no reason at all The audit trail cannot answer "why" for cancellations or rejections; kicks are traceable only when the organizer bothers to type a note. Evidence: 631.js:3627 <code>logAudit("match.cancelled", actorRef(t), { game: e.slice(-6) })</code> — the sanitized reason <code>r</code> is stored on the game (3617) but not in the audit entry; 631.js:3199-3235 <code>mockRejectParticipant</code> accepts no reason and the UI reject button (2466.js:711-716) has no prompt; kick note is optional (2466.js:876 <code>ve void 0</code>).	Weak evidence for dispute handling and abuse detection (e.g. discriminatory rejections).	Include <code>reason</code> in the <code>match.cancelled</code> audit payload, add an optional reason to reject (stored on the booking and audit), and require a note for kicks that trigger a refund.

ID / Severity	Interface	Risk Area	Identified Gap (with evidence)	Potential Impact	Recommended Remediation
F-ORG2-3 Low	organizer/series/[id]	Ambiguity	Cancelled series is stored as "ended" — the two outcomes are indistinguishable After a cancel (which nukes all occurrences) and after an end (which keeps them) the series shows the same "Ended" badge. The cancellation reason captured at 2471.js:210 is stored only on the games (cancellation_reason), never on the template. Evidence: 631.js:5800 — Qt.templates[o] = Object.assign({}, i, { status: "ended" }) inside mockCancelSeries; 909.js:2778-2780 only seriesStatus_active/paused/ended exist; 2471.js:46 badge tone falls to "danger" for anything not active/paused.	Organizer/admin cannot tell later whether a series was cancelled or naturally ended; the reason is lost at series level.	Add a cancelled template status with its own label, and persist cancelled_at / cancellation_reason on the template.

4. Interface-by-interface status

Admin: governance screens

- admin/application/[id] (1823)** — Shows one organizer application (risk flags, PII, identity, business, prior activity) with a free-text reason field and Approve / Request info / Reject / Suspend buttons; refreshes and shows "Done" after each action. *Partially works:* decisions execute and are audited, but reasons are optional, there is no confirmation, no self-review guard, "Request info" leads to an applicant dead end, and any load error is a bare "Error" page with no back button.
- admin/organizers (1832)** — Approval queue with status filters, four platform stats cards (NPN, growth, skill, compatibility) and a shortcut to players; refetches on focus. *Works*, with the caveat that the list is silently restricted to the admin's own audience world and stats do not reconcile with it.
- admin/players (1833)** — Player search with "move world" modal (reason required, works) plus a skill-intel roster with ± rating steppers. *Partially works:* audience correction is complete; rating adjustment has no error handling, confirmation or real reason, backend skips partition/existence checks, and refresh spam floods the capped audit log.
- admin/venues (1834)** — Platform financial tiles and a list of all venue registrations with Approve / Reject / Suspend and an inline commission editor; refreshes after each action. *Partially works:* mutations succeed and are audited, but there is no reason capture, no confirmation, no owner#reviewer guard (admin can approve and zero-commission their own venue), and financial tiles mix pending and settled amounts.
- admin/conduct (1826)** — Conduct stats and an incident queue with Approve/Reject (pending), Overturn (active) and Restore account (ban/suspension); refreshes after each action. *Partially works:* actions execute and are audited, but with no reason/confirmation, no success feedback, inert evidence links, a mislabelled "dangerous_play" category, and admin-issued bans never enter the queue.

Admin: analytics & configuration screens

- admin/awards (1824): lists collusion signals from award votes, links to /awards/[gameId]. Works as a read-only list, but errors render as "no fraud", the body copy is wrong, there is no review/dismiss workflow, and each visit spams the 500-entry audit log. Partially works.
- admin/concierge (1825): toggle + 8 steppers for platform concierge rules, plus learning stats. Works; saves are unguarded (no error feedback, lost updates on rapid taps), audit entry carries no diff/reason, error state shows organizer copy. Partially works.
- admin/demand (1827): 8 steppers for demand weights plus model stats. Works; same unguarded-save and audit gaps; the stats do not depend on the weights being tuned; NaN/mass-assignment possible server-side. Partially works.
- admin/feed (1828): read-only feed analytics and current ranking weights. Works; weights drift from user clicks with no admin control; any error shows "Access denied". Works (read-only).
- admin/funnel (1829): funnel event counts (7d/total) from the audit log. Works, but totals are capped by the 500-entry log, event names are client-supplied free text, and errors show as "no events". Partially works.
- admin/insights (1830): 9-tab BI dashboard with range/sport filters and CSV export; analyst has read-only access. Works; one failed fetch locks the screen, export silently substitutes Executive for Sports/Health tabs, no CSV-injection guard, simulated health metrics. Partially works.
- admin/optimizer (1831): dashboard (accuracy, importance, 3 bar charts) + 10 weight steppers. Works; importance bars go stale after saving, same unguarded-save/audit/NaN gaps. Partially works.
- (tabs)/profile admin panel (1805.js:752-822): shows Insights to admin+analyst and 9 further links to admin only; all targets exist. /admin/players is not listed (only via organizers header icon); section label is hard-coded "ADMIN". Works.

Organizer: onboarding & match creation

- organizer/apply (2459): Application form with status card. Works for first submission, approval and
- organizer/index (2465): Dashboard (reputation, stats, referrals, series, match tabs, ratings). Works and
- organizer/create (2461): Main creation screen with "quick"/"custom" lanes, optimizer, recurring
- organizer/new (2468): Full alternative wizard with drafts, formation and discard confirmation. Orphaned
- organizer/quick (2470): 3-step sport/venue/kickoff poster with client-side past-time check and inline
- organizer/smart-schedule (2472): Lists ranked slots and pushes to create with query params. Works;
- organizer/concierge (2460): Generates a single plan and pushes to create. Works; error alerts shown;

Organizer: match & series management

- organizer/match/[id] (2466.js) — Full organizer console: edit/cancel match, close/reopen registration, approve/reject/kick participants, attendance, score, Need-Player-Now, concierge auto-invite, smart replacement, group bookings, sanctions, activity log. **Works** end to end (all mutations reload via qe(), ownership enforced by zi on every mutation). Main gaps: unguarded approve after match end, editable past start time, in-progress cancel with forced refunds, no error handling on initial load, participant roster leaked to non-organizers by the screen endpoint, and sanction targeting not limited to the organizer's own participants.
- organizer/series/[id] (2471.js) — Series dashboard: analytics tiles, pause/resume/end/cancel, list of next 12 upcoming occurrences linking to organizer/match/[id]. **Partially works:** pause/resume/end/cancel function and are authorized server-side, but cancel does not refund paid seats or release courts, "End" has no confirmation and leaves occurrences live, analytics are unauthenticated estimates, and a bad id or fetch error leaves an infinite spinner with no back button. No series-level edit UI (backend editFutureOccurrences unused). All

route targets used (`/organizer/match/:id`, `/game/:id`, `/squad/:id`) exist in routes.txt.

5. Senior architect & security review

5.1 Security & compliance

Question	Assessment
Injection / XSS / CSRF	No SQL exists (no database). React Native Web escapes text; the 15 <code>dangerouslySetInnerHTML</code> and 5 <code>innerHTML</code> occurrences are inside framework style injection, not app code, and user text is rendered through <code>Text</code> components. User-supplied inputs are sanitised (<code>sanitizeName</code> , <code>sanitizeText</code> , <code>isValidEmail</code>) before storage. CSRF is not applicable while there is no server; when one is added, the Bearer-token RPC design is CSRF-safe but needs origin checks and rate limiting. The single-file build forces <code>script-src 'unsafe-inline'</code> , which weakens CSP against any future injection.
Broken authentication	Critical. Authentication is simulated in the browser (F-XC-1, F-XC-5, F-XC-9). Static OTP 123456 is accepted and displayed; password hashes use 12k-round SHA-256 on the client; sessions never expire and cannot be revoked; lockout state is client-editable. A hard-coded third-party auth URL is also present (F-XC-4).
Sensitive data handling	Organizer applications store ID-document references as SHA-256 hashes with a fixed pepper (<code>hashIdentifier</code> , 632.js:13-16), keep last-4 digits, and hash phone numbers, which is a reasonable minimisation design. However all of it, plus user emails and password hashes, sits in <code>localStorage</code> in clear JSON. The fixed pepper string is in the bundle. No encryption at rest, no retention policy, no deletion of applicant PII after decision.
Access control (RBAC)	Role model is simple and consistently checked in the mock (admin, analyst, organizer, per-game organizer, venue staff). Gaps: role changes are not themselves audited with actor and reason; there is no separation between "approver" and "operator" admins; no maker-checker for financial actions (commission, credit grants, refunds); no client-side route guard so non-admins reach admin URLs and see ambiguous empty states (F-XC-14). Everything is bypassable client-side until moved server-side.
Audit & non-repudiation	Critical. See F-XC-7. Placeholder IP, actor-editable, 500-row cap, no admin viewer.

5.2 Architecture & scalability

- Tight coupling.** One 16.5k-line module (631) contains every business rule for every domain (auth, games, payments, wallets, clubs, feed, awards, sanctions, BI). The UI layer is cleanly separated behind the facade (671), which is the right seam for a server migration, but the facade signatures pass raw user ids as the first argument (`fetchX(userId, ...)`), meaning the caller asserts its own identity. A server must derive identity from the token and ignore that argument.
- Bottlenecks.** Whole-collection reads and writes on every operation; full hydration at boot; no pagination in admin lists (players, applications, incident queue); statistics endpoints recompute over all rows on each screen focus. Fine for demo data, unusable at thousands of games.
- Concurrency.** Single-user by construction. Optimistic concurrency, idempotency keys and transactional writes (seat holds, payments, refunds) do not exist; the `withAdvisoryLock` hook is stubbed. Under concurrent load a real backend built by porting this code would double-book seats and double-refund.
- Error handling.** Two patterns are widespread: `catch {}` around fetches in screens (state becomes empty and the user sees a success-looking empty state) and `Za` swallowing persistence failures (F-XC-8). Both convert failures into silent data loss.
- Environment coupling.** Behaviour toggles on `process.env` values that are inlined at build time (`PLAYORA_DEMO`, `PLAYORA_ADMIN_EMAILS`, `PLAYORA_REMINDER_LEAD_MS`, SMS provider settings) and on runtime `localStorage` keys. There is no single configuration surface.

5.3 Code quality & maintainability

- Strengths.** Consistent screen skeleton (header, loading, empty, error, list), shared UI kit (Card, Badge, EmptyState, FormError), complete bilingual dictionary, accessibility roles and labels on pressables, RTL-aware chevrons, sanitisation helpers, and explicit error codes (`E_*`) mapped to localised messages.
- Debt.** (1) The mock store is a monolith that mixes seeding, demo fixtures and production rules; demo behaviour cannot be removed without touching real logic. (2) Three overlapping match-creation screens (create, new, quick) with diverging validation (see ORG1). (3) Duplicate stats endpoints with copy-pasted aggregation loops (five `mockGet*AdminStats` functions). (4) Magic ids for seeded admins (`...00ad`, `...00ae`, `...00a1`). (5) Numerous $O(n \cdot m)$ scans (`Qt.profiles.find` inside loops over games/participants) that would become $N+1$ queries if ported naively.
- Repository hygiene.** No source, no package manifest, no lint/type configuration, a byte-identical 11.6 MB duplicate file, and deploy config pointing at files that do not exist.

5.4 Testing & deployment readiness

- Testability.** The facade/store split makes unit testing of business rules easy once source is available (the mock store is pure JavaScript over an in-memory object). Screens use hooks and could be tested with React Native Testing Library. Today there are zero tests and no CI.
- Missing failure modes.** Storage quota exceeded; storage disabled (private browsing); clock skew for cut-off calculations; concurrent organizer actions on the same match; refund after partial attendance; admin approving an application after the applicant deleted their account; deep link to an admin route as guest; RTL layout of the admin tables; export on web (no file download path in a `connect-src 'none'`, `data:`-restricted page).
- Deployment.** Not deployable from this repository (F-XC-10). Minimum viable path: Railway static service serving `index.html`, then a separate API service. Recommended path: commit the Expo project, build in CI, and publish the web export plus an API.

6. Prioritised remediation plan

Phase	Actions	Findings addressed
-------	---------	--------------------

0. Stop-gap (days)	Label the site as a demo; remove seeded admins, static OTP and auto-approved organizer behind an explicit <code>EXP0_PUBLIC_DEMO</code> flag; fix Railway config to serve the static file; delete the duplicate HTML; commit the Expo source.	F-XC-5, F-XC-6, F-XC-10, F-XC-11
1. Screen fixes (1-2 weeks)	Fix the screen-level bugs and dead ends in section 3 (ADM1/ADM2/ORG1/ORG2): stale data after mutations, swallowed errors, missing confirmations and reason capture, unreachable routes, misleading empty states; add an admin route guard.	ADM*/ORG* findings, F-XC-14
2. Real backend (4-8 weeks)	Implement the <code>/rpc</code> contract server-side (Node + PostgreSQL), derive identity from the token, port the role checks (<code>sn</code> , <code>Ms</code> , <code>on</code> , <code>organizer-of-game</code>) as middleware, move persistence to the database with transactions and idempotency keys for payments/seat holds, and set <code>CSP connect-src</code> to the API origin.	F-XC-1, F-XC-2, F-XC-3, F-XC-4, F-XC-8, F-XC-9, F-XC-13
3. Controls (with phase 2)	Server-side append-only audit log with admin viewer and export; mandatory reason on <code>approve/reject/sanction/commission/credit</code> ; maker-checker on financial actions; PII retention and deletion for applications; role-change auditing; rate limiting on auth endpoints.	F-XC-7 and control gaps in ADM1/ORG2
4. Quality (ongoing)	CI with typecheck, lint, unit tests for store rules, Playwright smoke tests for admin and organizer flows; split the 631 monolith by domain; normal Expo web export with cached assets and lazy admin routes.	F-XC-12, maintainability items in 5.3

7. Appendix A: Route-to-module map

The bundle was split at each `__d(function ...)` Metro module boundary and prettified with Prettier. Line numbers in the evidence column refer to these prettified module files (available on request as `pretty/<id>.js`). The route table below was recovered from the Expo Router context module (id 18).

Screens in scope

Route file	Module id
<code>./(auth)/_layout.tsx</code>	19
<code>./(auth)/sign-in.tsx</code>	613
<code>./(auth)/sign-up.tsx</code>	917
<code>./(tabs)/profile.tsx</code>	1805
<code>./_layout.tsx</code>	1809
<code>./admin/application/[id].tsx</code>	1823
<code>./admin/awards.tsx</code>	1824
<code>./admin/concierge.tsx</code>	1825
<code>./admin/conduct.tsx</code>	1826
<code>./admin/demand.tsx</code>	1827
<code>./admin/feed.tsx</code>	1828
<code>./admin/funnel.tsx</code>	1829
<code>./admin/insights.tsx</code>	1830
<code>./admin/optimizer.tsx</code>	1831
<code>./admin/organizers.tsx</code>	1832
<code>./admin/players.tsx</code>	1833
<code>./admin/venues.tsx</code>	1834
<code>./organizer/apply.tsx</code>	2459
<code>./organizer/concierge.tsx</code>	2460

Shared modules referenced

Module id	Purpose
630	AuthProvider / <code>useAuth</code> (session state, sign-in/sign-up/guest)
631	Mock store: all business rules, role checks, seeding, persistence (16.5k lines prettified)
632	Password hashing (scrypt when available, else 12k-round SHA-256)
637	"Secure" store wrapper over AsyncStorage
642	SMS provider (log / HTTP)
643	Audit log (localStorage, 500 rows) and persistence health counters
671	API facade: <code>fetchX/mutateX</code> -> <code>store.mockX</code>
672	Store proxy: routes every <code>mockX</code> through <code>rpc()</code> with local fallback
673	Backend mode detection, <code>/health</code> probe, <code>/rpc</code> transport, session token
674	Store error code localisation
909	i18n dictionary (en/ar, 3,226 keys each)
913	Remote auth client (email, OTP, reset); hard-coded Neon Auth URL

Repository files referenced

File	Role
<code>index.html</code>	Compiled bundle (11.6 MB). Line 5 = CSP, line 30 = inlined assets, lines 130-2773 = JS bundle
<code>playora-standalone.html</code>	Byte-identical duplicate of <code>index.html</code>
<code>railway.json</code> , <code>nixpacks.toml</code>	Railway deploy configuration (references missing files)

<code>./organizer/create.tsx</code>	2461
<code>./organizer/index.tsx</code>	2465
<code>./organizer/match/[id].tsx</code>	2466
<code>./organizer/new.tsx</code>	2468
<code>./organizer/quick.tsx</code>	2470
<code>./organizer/series/[id].tsx</code>	2471
<code>./organizer/smart-schedule.tsx</code>	2472

<code>rpc.ts, auth/jwt.ts, .env.example</code>	Unused TypeScript helpers and env documentation
--	---

8. Appendix B: Method and limitations

- Static review only. The bundle was not executed against a live backend; behaviours were traced by reading the code paths. Where a finding depends on runtime state it is marked "needs confirmation".
- Source maps and original TypeScript were not available, so identifiers are minified (for example `sn` = `assert-admin`, `on` = `assert-approved-organizer`, `Za` = `persist collection`, `Qt` = `in-memory database`). Function names in the evidence follow the prettified dump.
- Screens outside the admin and organizer areas (player, venue portal, teams, wallet, media) were not audited in depth; cross-cutting findings in section 3 apply to them as well.
- Translation completeness was checked mechanically: all 3,226 keys exist in both `en` and `ar`; two apparent calls in `organizer/create` (`t("auto")`, `t("public")`) turned out to be state comparisons, not translation lookups, so no missing keys were found.